

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
Федеральное государственное автономное образовательное учреждение высшего
образования
**«Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского»
(ННГУ)**

**Институт информационных технологий, математики и механики
Кафедра: «Высокопроизводительных вычислений и системного
программирования»**

Направление подготовки: «Прикладная математика и информатика»
Магистерская программа: «Вычислительные методы и суперкомпьютерные технологии»

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ
на тему:
**«Создание инструмента для эмуляции исполнения CUDA RTX ядер на
центральном процессоре»**

Выполнил: студент группы
3824М1ПМвм
Мусин А.Н. _____

Научный руководитель:
доцент каф. ВВСП, к.т.н.
Горшков А.В. _____

Нижний Новгород
2026

Содержание

1	Введение	5
1.1	Актуальность и мотивация исследования	5
1.2	Анализ существующих решений	5
1.3	Цели и задачи	6
1.4	Научная новизна	6
1.5	Практическая значимость	7
2	Теоретические основы	8
2.1	PTX как промежуточное представление CUDA	8
2.1.1	Место PTX в стеке компиляции CUDA	8
2.1.2	Система типов PTX	9
2.1.3	Классификация инструкций PTX	9
2.1.4	Директивы PTX и их роль в эмуляторе	10
2.1.5	Пространства адресов и их семантика	10
2.2	Математическая модель SIMT-исполнения	11
2.2.1	Варп как алгебраическая структура	11
2.2.2	Маска исполнения: операции и инварианты	11
2.2.3	Предикатные регистры и составная маска	12
2.3	Поведение инструкций PTX	12
2.3.1	Обозначения	12
2.3.2	Арифметические инструкции	12
2.3.3	Инструкции сравнения и выбора	13
2.3.4	Инструкции управления потоком	13
2.3.5	Инструкции памяти	13
2.4	Механизм динамической подстановки библиотек в Linux	13
2.4.1	Загрузчик ELF и таблица динамических символов	13
2.5	Обзор смежных работ	14
2.5.1	Функциональные эмуляторы: Ocelot, CUDAsim	14
2.5.2	Микроархитектурный симулятор GPGPU-Sim	14
3	Механизм перехвата CUDA Runtime	15
3.1	Архитектура перехвата через LD_PRELOAD	15
3.1.1	Схема взаимодействия компонентов	15
3.1.2	Реализованные точки перехвата CUDA Runtime API	15
3.1.3	Извлечение PTX-текста через cuobjdump	16
3.1.4	Регистрация функций: отображение дескриптор → имя ядра	16
3.1.5	Запуск cuemi: утилита командной строки и переменные окружения	17
4	Синтаксический анализатор PTX	18
4.1	Требования к анализатору и выбор стратегии	18
4.1.1	Анализ структуры PTX-текста	18
4.1.2	Однопроходный инкрементный разбор	18
4.2	Реализация анализатора	19
4.2.1	Разбор объявлений функций и параметров	19
4.2.2	Таблица меток и граф базовых блоков	19
4.2.3	Разбор отдельной инструкции	19
4.2.4	Разрешение операндов	20
4.3	Внутреннее представление инструкций	21
4.3.1	Иерархия классов инструкций и монотонный счётчик PC	21

5	Система исполнения инструкций	23
5.1	Таблица диспетчеризации	23
5.1.1	Структура: фабрика и виртуальный вызов	23
5.1.2	Генерация кода классов инструкций	23
5.2	Регистровый файл и разрешение операндов	24
5.2.1	Структура регистрового файла	24
5.2.2	Вспомогательные шаблоны <code>reg_cast<T></code> и <code>to_u64<T></code>	24
5.2.3	Вычисление эффективного адреса	25
5.3	Исполнители арифметических инструкций	25
5.3.1	Целочисленная арифметика	25
5.3.2	Арифметика с плавающей запятой: IEEE 754 и режимы округления	26
5.3.3	Инструкции преобразования типов	26
5.4	Исполнители инструкций работы с памятью	26
5.4.1	<code>ld.global / ld.shared</code> : чтение	26
5.4.2	<code>st.global / st.shared</code> : запись с учётом маски	27
5.4.3	<code>setp / selp</code> : операции с предикатами	27
5.5	Исполнители инструкций управления потоком	27
5.5.1	Безусловный переход <code>bra.uni \$L</code>	27
5.5.2	Условный переход <code>@%p bra \$L</code>	28
5.5.3	<code>bar.sync</code> : делегирование барьерной синхронизации	28
6	Механизм управления дивергентным потоком	30
6.1	Теория PDOM-реконвергенции	30
6.1.1	Граф управляющего потока и постдоминаторы	30
6.1.2	Теорема о корректности PDOM-реконвергенции	30
6.2	Структура данных стека дивергенции	31
6.2.1	Тип <code>StackFrame</code> и семантика полей	31
6.2.2	Сложность операций и ограничение глубины	31
6.3	Алгоритм обработки условного перехода	32
6.3.1	Алгоритм PDOM_Diverge	32
6.3.2	Обнаружение IPDOM и цикл реконвергенции	32
6.4	Граничные случаи	33
6.4.1	Вложенные дивергенции: корректность при $k \geq 2$	33
6.4.2	Дивергентные циклы	34
6.4.3	Инструкция <code>exit</code> и обнуление маски активных потоков	34
7	Система профилирования	35
7.1	Архитектура профилировщика	35
7.1.1	Хуки профилирования с автогенерацией кода	35
7.1.2	Структура <code>ProfilingRecord</code> и буферизация	36
7.1.3	Метка времени: <code>steady_clock</code>	36
7.2	Формальное определения метрик производительности	37
7.2.1	<code>branch_efficiency</code> : эффективность ветвлений	37
7.2.2	<code>bank_conflicts</code> : конфликты банков разделяемой памяти	37
7.2.3	<code>global_coalescing</code> : коэффициент слияния транзакций	37
7.2.4	<code>write_ops</code> : давление на регистровый файл	38
7.3	Инструмент генерации отчётов	38
7.3.1	Анализатор текстового вывода (<code>parser.py</code>)	38
7.3.2	PTX-анализатор для отчёта (<code>ptx_parser.py</code>)	38
7.3.3	Агрегатор (<code>aggregator.py</code>): формулы накопления	39

7.3.4	Рендерер HTML-отчёта (<code>renderer.py</code>)	39
8	Тестирование и верификация	41
8.1	Модульные тесты	41
8.2	Часть I — Интеграционное тестирование CUDA-ядер	41
8.2.1	<code>vadd_custom</code> — поэлементное сложение векторов	41
8.2.2	<code>gelu_custom</code> — функция активации GELU	42
8.2.3	<code>softmax_custom</code> — онлайн-нормализация Softmax	42
8.2.4	<code>mmul_custom</code> — тайловое матричное умножение	43
8.2.5	<code>attention_custom</code> — Flash Attention-подобное ядро	43
8.2.6	<code>fft_custom</code> — Radix-2 DIT FFT, $N = 32$	44
8.2.7	<code>conjugate_gradient</code> — итерационное решение СЛАУ	44
8.2.8	Сводная таблица результатов интеграционных тестов	44
8.3	Часть II — Тестирование системы профилирования	45
8.3.1	Точность метрик	45
8.3.2	Привязка инструкций к исходному коду	45
8.3.3	Сквозная проверка цепочки профилировщика	46
9	Заключение	47
A	Таблица поддерживаемых инструкций PTX	48
B	Поведение инструкций PTX: полный список правил	50
C	Трассировка PDOM_Diverge на примере вложенной дивергенции	52
D	Формат файла <code>profiling.txt</code>: аннотированный пример	54
E	Структура HTML-отчёта: описание секций	55
F	Репозиторий с исходным кодом	57

1 Введение

1.1 Актуальность и мотивация исследования

Графические ускорители класса NVIDIA GPU заняли центральное место в современных высокопроизводительных вычислениях [? ?]: от обучения крупных языковых моделей до физического моделирования и научных расчётов. Программирование GPU на платформе CUDA строится на модели SIMT (Single Instruction, Multiple Threads), в которой группы из 32 потоков — варпы — исполняют одну и ту же инструкцию над различными данными [? ?]. Ключевым промежуточным представлением в данной экосистеме служит PTX (Parallel Thread Execution) — аппаратно-независимый ассемблер NVIDIA [?], в который компилируется CUDA C++ и из которого генерируется машинный код SASS для конкретного поколения GPU.

Несмотря на широкое распространение CUDA, все официальные инструменты отладки и профилирования — Nsight Compute, Nsight Systems, Compute Sanitizer — требуют наличия физического GPU. Это делает их недоступными в облачных CI-средах, виртуальных машинах и академических лабораториях с ограниченным бюджетом.

Программный эмулятор PTX ISA снимает это ограничение: CUDA-ядра исполняются на стандартном CPU, а встроенный профилировщик собирает метрики с точностью до отдельной инструкции. Это определяет следующие области применения:

1. **Среды непрерывной интеграции без GPU.** Автоматическая проверка корректности CUDA-ядер при каждом коммите без специализированного оборудования.
2. **Академические исследования.** Изучение архитектурных свойств GPU-программ без доступа к аппаратным счётчикам.
3. **Разработка и отладка алгоритмов.** Детерминированное пошаговое исполнение с полным контролем над состоянием варпа.
4. **Образовательные задачи.** Наглядное представление механизмов дивергенции, слияния транзакций памяти и конфликтов банков для изучения принципов GPU-архитектуры.

1.2 Анализ существующих решений

На сегодняшний день существует ряд инструментов, частично решающих задачу функциональной эмуляции CUDA-ядер.

Ocelot [?] — один из первых функциональных PTX-интерпретаторов с поддержкой перехвата через CUDA Runtime API. Его ключевое ограничение — привязка к PTX ISA версий 1.x–2.x (CUDA 4.x) и отсутствие поддержки директив PTX 7.x+, в частности инструкций `ldmatrix`, `wmma` и атрибута `.address_size 64`. Проект не поддерживается с 2013 года.

GPGPU-Sim [?] — timing-accurate микроархитектурный симулятор. Поддерживает PTX ISA только до версии 4.x и ориентирован на оценку латентностей, а не функциональную корректность.

Compute Sanitizer — официальный инструмент NVIDIA для обнаружения гонок данных, ошибок памяти и нарушений барьерной синхронизации. Требует наличия физического GPU и не предоставляет метрик уровня инструкций PTX.

Актуального PTX-эмулятора, поддерживающего ISA 7.x+, с интегрированным профилировщиком и прозрачным перехватом CUDA Runtime API без модификации бинарного файла, не существует — этот пробел и заполняет данная работа.

1.3 Цели и задачи

Цель работы — разработка программного эмулятора PTX ISA, обеспечивающего функционально корректное исполнение CUDA-ядер без GPU с возможностью сбора детальных метрик производительности на уровне инструкций.

Выбор PTX ISA в качестве уровня эмуляции обусловлен тем, что PTX является стабильным публичным контрактом NVIDIA [?]: компилятор гарантирует его синтаксическую и семантическую совместимость между поколениями GPU, тогда как SASS — бинарный машинный код — документируется значительно хуже и изменяется с каждым поколением архитектуры.

Для достижения указанной цели поставлены следующие **задачи**:

1. Реализовать механизм прозрачного перехвата CUDA Runtime API через LD_PRELOAD с извлечением PTX посредством cuobjdump.
2. Разработать однопроходный инкрементный синтаксический анализатор PTX-текста, поддерживающий подмножество PTX ISA 7.x.
3. Построить таблицу диспетчеризации и исполнители инструкций с формальным описанием поведения инструкций.
4. Реализовать алгоритм PDOM-реконвергенции для управления дивергентным потоком варпа.
5. Разработать систему метрик профилировщика с автогенерацией исполнителей из JSON файла.
6. Реализовать инструмент генерации самодостаточного HTML-отчёта с аннотацией строк исходного кода.
7. Верифицировать корректность эмулятора на семи интеграционных тестах с реальными CUDA-ядрами.

1.4 Научная новизна

Научная новизна работы заключается в следующем.

1. Впервые реализован прозрачный механизм перехвата CUDA Runtime API посредством LD_PRELOAD без модификации целевого бинарного файла с поддержкой

PTX ISA 7.x; дано формальное обоснование порядка разрешения символов динамическим загрузчиком.

2. Разработан и верифицирован однопроходный синтаксический анализатор подмножества PTX ISA 7.x, корректность которого подтверждена на семи CUDA-ядрах и тридцати одном модульном тесте.
3. Описано поведение инструкций подмножества PTX ISA в виде формальных правил, позволяющих верифицировать корректность исполнителей без обращения к аппаратному обеспечению.
4. Предложена архитектура расширяемой системы сбора метрик с автогенерацией исполнителей из JSON-файла, при которой добавление новой метрики не требует модификации кода на C++.
5. Получена формальная модель коэффициента слияния обращений к глобальной памяти $\kappa = T_{\text{ideal}}/T_{\text{actual}}$ с учётом размера элемента доступа.

1.5 Практическая значимость

Разработанный инструмент имеет непосредственное практическое применение:

- **Внедрение в конвейеры непрерывной интеграции (CI).** Эмулятор запускается как обычный Linux-процесс; для включения в `GitHub Actions` или `Jenkins` достаточно однострочного скрипта `cuemu ./binary`.
- **Детальное профилирование.** HTML-отчёт с аннотацией строк исходного кода позволяет определить «горячие» инструкции и оптимизировать шаблоны обращения к памяти без запуска на GPU [?].
- **Образовательная ценность.** Пошаговая трассировка маски дивергенции обеспечивает наглядную демонстрацию механизма SIMT-исполнения.

Инструмент не требует специализированного оборудования: PTX извлекается утилитой `cuobjdump` из набора NVIDIA, ядра исполняются полностью на CPU.

2 Теоретические основы

2.1 PTX как промежуточное представление CUDA

2.1.1 Место PTX в стеке компиляции CUDA

Компиляция CUDA-программы проходит через многоуровневый конвейер. На входе компилятор `nvcc` [?] принимает файл `.cu`, содержащий смешанный код: C++-код хоста и тела ядер устройства. Внешний интерфейс компилятора выделяет код устройства и передаёт его в NVVM — LLVM-based IR [?] для GPU — который транслируется в PTX.

PTX (Parallel Thread Execution) — аппаратно-независимое промежуточное представление, описывающее программу в терминах виртуального RISC-процессора с бесконечным регистровым файлом и явной параллельностью потоков. На следующем шаге `ptxas` транслирует PTX в SASS — бинарный машинный код конкретной архитектуры (Volta, Turing, Ampere, Hopper), который помещается в fatbinary-архив вместе с исходным PTX-текстом. При запуске CUDA Runtime JIT-компилирует PTX для установленного GPU при отсутствии подходящего SASS.

Fatbinary является ELF-совместимым архивом, встраиваемым компоновщиком в секцию `.nv_fatbin` результирующего ELF-файла приложения. Эмулятор извлекает PTX-текст из этой секции при перехвате `__cudaRegisterFatBinary`, вызывая утилиту `cuobjdump -dump-ptx` (подробно — в Главе 3).

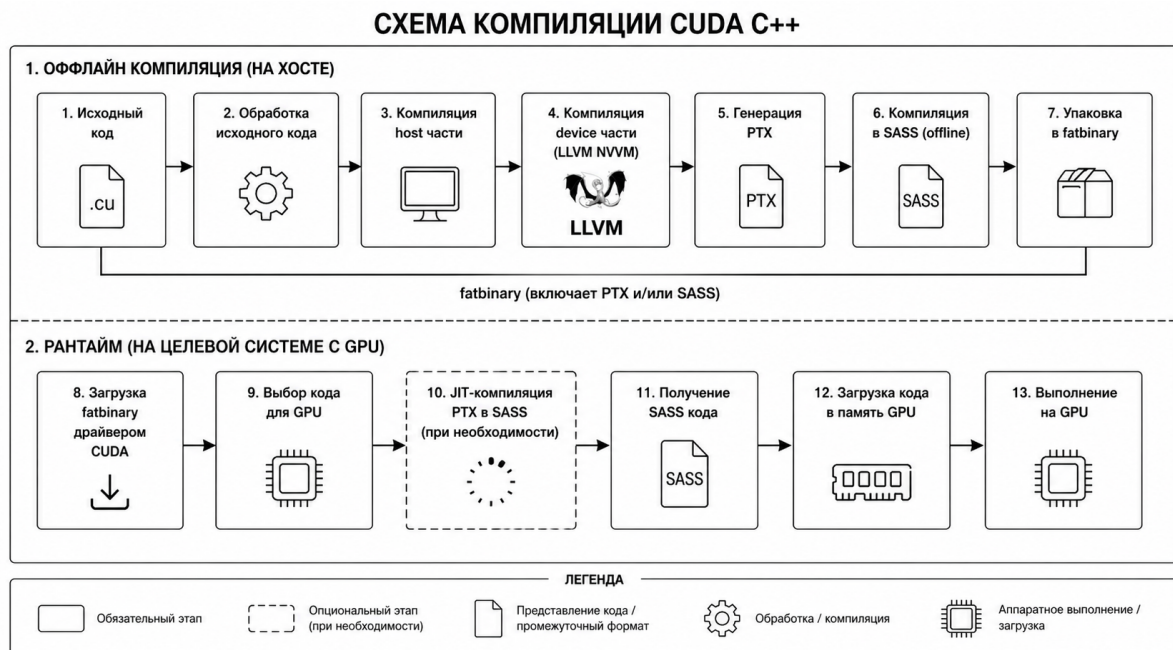


Рисунок 1: Стек компиляции CUDA. PTX занимает уровень между платформо-независимым IR и машинным кодом SASS.

2.1.2 Система типов РТХ

РТХ определяет следующие базовые типы данных. Тип операнда определяется двумя способами: суффиксом инструкции (`.f32`, `.s32` и др.) и префиксом имени регистра (`%f` $\rightarrow$ `f32`, `%r` $\rightarrow$ `s32`, `%rd` $\rightarrow$ `u64`, `%p` $\rightarrow$ `pred`):

- **Целочисленные знаковые:** `s8`, `s16`, `s32`, `s64` — арифметика в дополнительном коде.
- **Целочисленные беззнаковые:** `u8`, `u16`, `u32`, `u64`.
- **Битовые:** `b8`, `b16`, `b32`, `b64`, `b128` — интерпретация не задана, используются для пересылок и упаковки.
- **Вещественные:** `f16`, `f32`, `f64` — форматы IEEE 754-2008 [?] половинной, одинарной и двойной точности.
- **Предикат:** `pred` — однобитное логическое значение, хранимое в регистрах `%p0–%pN`.

Суффикс типа однозначно задаёт битовую ширину $w \in \{8, 16, 32, 64, 128\}$ и способ интерпретации битового шаблона.

Различие знаковых, беззнаковых и битовых вариантов одной ширины непосредственно влияет на реализацию исполнителей: `shr.u32` выполняет логический сдвиг, тогда как `shr.s32` — арифметический (с распространением знакового бита).

2.1.3 Классификация инструкций РТХ

Для целей проектирования таблицы диспетчеризации все инструкции РТХ разбиты на семь классов:

1. **Арифметические** (`add`, `sub`, `mul`, `div`, `fma`, `mad`, `rem`, `abs`, `neg`, `min`, `max`) — 25 мнемоник.
2. **Логические и сдвиги** (`and`, `or`, `xor`, `not`, `shl`, `shr`, `bfe`, `bfi`, `popc`) — 14 мнемоник.
3. **Сравнения и предикаты** (`setp`, `selp`, `slct`, `vote`) — 4 мнемоники.
4. **Работа с памятью** (`ld`, `st`, `mov`, `cvta`, `prefetch`) — 5 мнемоник.
5. **Преобразование типов** (`cvt`) — 1 мнемоника с матрицей суффиксов 5×5 .
6. **Управление потоком** (`bra`, `call`, `ret`, `exit`, `trap`) — 5 мнемоник.
7. **Синхронизация и специальные** (`bar`, `membar`, `mov.special`, `ex2`, `lg2`, `rcp`, `sqrt`, `sin`, `cos`) — 9 мнемоник.

2.1.4 Директивы PTX и их роль в эмуляторе

В PTX-тексте директивы не являются исполняемыми инструкциями, но несут информацию, критически необходимую для анализатора:

- `.entry / .func` — задают тип функции (ядро устройства или вспомогательная функция) и её границы.
- `.param` — описывают сигнатуру: имя, тип и выравнивание каждого параметра ядра.
- `.reg .type %prefix<N>` — объявляют регистровый файл: например, `.reg .f32 %f<115>` резервирует 115 регистров типа `f32` с именами `%f0–%f114`.
- `.shared .align A .b8 name[S]` — резервирует `S` байт разделяемой памяти с выравниванием `A` байт.
- `.file ID "path"` и `.loc ID L C` — формируют таблицу, привязывающую PTX-инструкции к строкам исходного кода, используемую профилировщиком для аннотации отчёта.

2.1.5 Пространства адресов и их семантика

PTX определяет шесть пространств адресов, отличающихся по видимости и времени жизни:

- `.reg` — регистровый файл, приватный для каждого потока.
- `.global` — глобальная память, разделяемая всеми блоками.
- `.shared` — разделяемая память, видимая всем потокам одного блока; время жизни — время жизни блока.
- `.local` — приватная память стека потока (спилл регистров).
- `.param` — область передачи параметров ядра с хоста.
- `.const` — константная память, только для чтения.

Формально вводится функция пространства: $space : Addr \rightarrow \{global, shared, local, param, const\}$, используемая исполнителями `ld` и `st` для выбора соответствующего обработчика обращения к памяти.

Смешение пространств в реализации исполнителей недопустимо: глобальная и разделяемая память обслуживаются разными аппаратными блоками с различными правилами доступа, что непосредственно влияет на вычисление метрик слияния транзакций и конфликтов банков.

2.2 Математическая модель SIMT-исполнения

2.2.1 Варп как алгебраическая структура

В модели SIMT все потоки варпа в каждый момент времени исполняют одну и ту же инструкцию, однако каждый поток имеет *собственный счётчик команд и регистровый файл*. Это допускает дивергенцию: при условном переходе потоки, для которых условие выполнено, и потоки, для которых оно не выполнено, временно расходятся по разным ветвям, а аппаратура отслеживает активность каждого потока через маску исполнения и стек реконвергенции.

Базовой единицей исполнения на GPU является *varp* — группа из 32 потоков, всегда исполняющих одну и ту же инструкцию [? ?]. Формально варп определяется как кортеж:

$$W = \langle T, M, PC, R, S \rangle, \quad (1)$$

где:

- $T = \{0, 1, \dots, 31\}$ — множество идентификаторов потоков варпа;
- $M \in \mathbb{B}^{32}$ — маска исполнения (execution mask): $M[i] = 1$ означает, что поток i исполняет целевую инструкцию;
- $PC \in \mathbb{N}$ — программный счётчик;
- R — регистровый файл (функция $T \times Types \times \mathbb{N} \rightarrow Val$);
- S — стек дивергенции (подробно в Главе 6).

Операция исполнения одной инструкции определяется как функция перехода состояния: $\delta : W \times Instr \rightarrow W'$.

2.2.2 Маска исполнения: операции и инварианты

Маска $M \in \mathbb{B}^{32}$ формально задаёт, какие потоки исполняют следующую инструкцию. Определим вспомогательные операции:

$$\text{active}(M) = \{i \in T \mid M[i] = 1\}, \quad (2)$$

$$\text{popcount}(M) = |\text{active}(M)|, \quad (3)$$

$$\text{apply}(M, M') = M \wedge M' \quad (\text{маскирование по предикату}). \quad (4)$$

Фундаментальный инвариант исполнения: *исполнитель инструкции обязан игнорировать потоки с $M[i] = 0$ при записи результата*. Нарушение данного инварианта порождает логические ошибки в реализации исполнителей и строго контролируется в тестах.

В отличие от наивного однопоточного исполнителя, потоки с $M[i] = 0$ не просто пропускают инструкцию — они не должны изменять регистровый файл и память, иначе результат разойдётся с аппаратным SIMT-исполнением.

2.2.3 Предикатные регистры и составная маска

Предикатный регистр $P \in \mathbb{B}^{32}$ хранит по одному биту на каждый поток варпа. Инструкция `@%p instr` выполняется с эффективной маской:

$$M_{\text{eff}} = M \wedge P, \quad (5)$$

а инструкция `@!%p instr` — с:

$$M_{\text{eff}} = M \wedge \neg P. \quad (6)$$

Инструкция `setp.op.type %p, a, b` вычисляет:

$$P[i] = op(a_i, b_i), \quad \forall i \in \text{active}(M), \quad (7)$$

что при следующем условном переходе `@%p bra` может привести к расщеплению варпа, если $M \wedge P \neq 0$ и $M \wedge \neg P \neq 0$.

2.3 Поведение инструкций RTX

2.3.1 Обозначения

Состояние варпа описывается тремя компонентами:

- $R[t][\tau][k]$ — значение k -го регистра типа τ потока t ;
- M — 32-битная маска исполнения (бит t равен 1, если поток t активен);
- PC — номер текущей инструкции.

Запись $\forall t \in \text{active}(M)$ означает, что правило применяется только к активным потокам; регистры неактивных потоков не изменяются.

2.3.2 Арифметические инструкции

`add.s32 %rd, %ra, %rb` складывает два 32-битных целых и записывает результат в `%rd`, отбрасывая переполнение:

$$R'[t][s32][d] = (R[t][s32][a] + R[t][s32][b]) \bmod 2^{32}, \quad \forall t \in \text{active}(M). \quad (8)$$

`fma.rn.f32 %fd, %fa, %fb, %fc` вычисляет $a \cdot b + c$ за одну операцию с одним округлением по стандарту IEEE 754 [?] (режим «к ближайшему чётному»):

$$R'[t][f32][d] = \text{round}_n(R[t][f32][a] \cdot R[t][f32][b] + R[t][f32][c]), \quad \forall t \in \text{active}(M). \quad (9)$$

2.3.3 Инструкции сравнения и выбора

`setp.lt.s32 %p, %ra, %rb` записывает результат сравнения в предикатный регистр `%p`:

$$P'[t] = (R[t][s32][a] < R[t][s32][b]), \quad \forall t \in \text{active}(M). \quad (10)$$

`sefp.f32 %fd, %fa, %fb, %p` выбирает одно из двух значений в зависимости от предиката, не вызывая дивергенцию:

$$R'[t][f32][d] = \begin{cases} R[t][f32][a] & \text{если } P[t] = 1, \\ R[t][f32][b] & \text{если } P[t] = 0, \end{cases} \quad \forall t \in \text{active}(M). \quad (11)$$

2.3.4 Инструкции управления потоком

`bra.uni $L` — безусловный переход: все потоки варпа переходят к метке `$L`, маска M не меняется:

$$PC' = \text{offset}(\$L), \quad (12)$$

где $\text{offset}(\$L)$ — номер первой инструкции метки `$L` из таблицы базовых блоков.

`@%p bra $L` — условный переход. Обозначим $M_{\text{taken}} = M \wedge P$ (потоки, берущие переход) и $M_{\text{not}} = M \wedge \neg P$ (остальные):

- Если $M_{\text{not}} = 0$: все активные потоки берут переход, $PC' = \text{offset}(\$L)$.
- Если $M_{\text{taken}} = 0$: ни один поток не берёт переход, $PC' = PC + 1$.
- Иначе: часть потоков расходится — применяется алгоритм `PDOM_Diverge` (подробно в Главе 6).

2.3.5 Инструкции памяти

`ld.global.f32 %fd, [%ra]` читает 4-байтное значение из глобальной памяти по адресу в регистре `%ra`:

$$R'[t][f32][d] = \text{Mem}_{\text{global}}[R[t][u64][a]], \quad \forall t \in \text{active}(M). \quad (13)$$

После исполнения `ld.global/st.global` вычисляются T_{actual} и κ по формулам (30)–(32).

2.4 Механизм динамической подстановки библиотек в Linux

2.4.1 Загрузчик ELF и таблица динамических символов

При запуске ELF-бинарного файла в Linux ядро передаёт управление динамическому загрузчику `ld.so`. Загрузчик строит *список разрешения символов* — упорядоченную цепочку библиотек, в которой он ищет адрес каждого используемого символа [? ?].

При помещении `libemuruntime.so` на позицию l_1 (через `LD_PRELOAD=libemuruntime.so`) все вызовы `cudaMalloc`, `cudaLaunchKernel` и прочих функций CUDA Runtime разрешаются в реализации эмулятора согласно правилу первого совпадения в цепочке разрешения символов, не достигая оригинальной `libcudart.so`.

2.5 Обзор смежных работ

2.5.1 Функциональные эмуляторы: Ocelot, CUDAsim

Проект Ocelot [?] разработан в Технологическом институте Джорджии в 2009–2013 гг. Он реализует функциональную эмуляцию PTX с поддержкой перехвата CUDA 4.x API через механизм, аналогичный `LD_PRELOAD`. Ocelot не поддерживает PTX ISA 7.x+ (инструкции `ldmatrix`, `wmma`, `async.bulk.copy`) и не компилируется с современными версиями GCC/Clang. Возможности профилирования ограничены счётчиком выполненных инструкций.

CUDAsim (CUDA Device Emulation, устаревший режим) — встроенный в CUDA 4.x режим эмуляции CPU, удалённый в CUDA 5.0 из-за расхождений в поведении атомарных операций и барьеров.

2.5.2 Микроархитектурный симулятор GPGPU-Sim

GPGPU-Sim [?] — timing-accurate симулятор, воспроизводящий конвейер SM с точностью до цикла: внешний интерфейс, исполнение инструкций, исполнительные блоки, кэши, DRAM. Поддерживаемое подмножество PTX ISA ограничено версиями до 4.x; инструкции PTX 7.x+ (`ldmatrix`, `wmma`, атрибут `.address_size 64`) не поддерживаются. Его цель принципиально отличается от задачи данной работы: оценка латентностей и пропускной способности, а не функциональная корректность в контексте непрерывной интеграции. Один запуск `mmul_custom (64×64×64)` в GPGPU-Sim занимает порядка 10 часов машинного времени.

Сравнение подходов приведено в таблице 1.

Таблица 1: Сравнение инструментов эмуляции/симуляции CUDA

Инструмент	PTX ISA	Перехват	Профилировщик	Тип
Ocelot	1.x–2.x	LD_PRELOAD	Счётчики	Функциональный
GPGPU-Sim	≤ 4.x	Не требуется	Timing	Архитектурный
Compute Sanitizer	—	Требует GPU	Нет	Аппаратный
Данная работа	7.x	LD_PRELOAD	4 метрики	Функциональный

3 Механизм перехвата CUDA Runtime

3.1 Архитектура перехвата через LD_PRELOAD

3.1.1 Схема взаимодействия компонентов

Центральная идея механизма перехвата состоит в том, что CUDA-приложение вызывает функции из `libcudart.so` через PLT, а динамический загрузчик `ld.so` [? ? ?] разрешает эти символы в реализации из `libemuruntime.so`, заблаговременно помещённой в переменную `LD_PRELOAD`.

Полная схема взаимодействия компонентов:

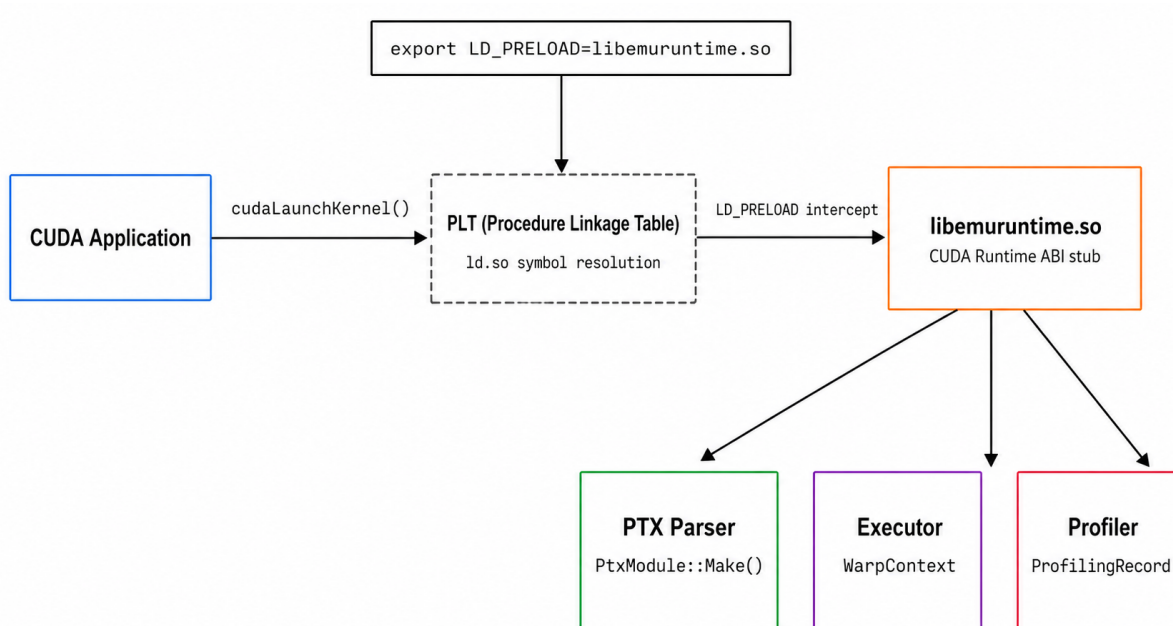


Рисунок 2: Схема перехвата CUDA Runtime через LD_PRELOAD.

Библиотека `libemuruntime.so` реализует набор CUDA Runtime API вызовов, достаточный для работы типичного CUDA-приложения. Каждая функция-заглушка либо делегирует работу компонентам эмулятора (синтаксическому анализатору, исполнителю, профилировщику), либо возвращает `cudaSuccess` для функций, не требующих реализации в рамках эмулятора (например, управление событиями, профили потоков).

Большинство ядер не проверяют статус возврата управляющих вызовов (`cudaEventCreate` и аналогичных), поэтому возврат `cudaSuccess`-заглушки достаточен для корректного исполнения вычислительного ядра.

3.1.2 Реализованные точки перехвата CUDA Runtime API

В таблице 2 перечислены восемь перехватываемых функций CUDA Runtime API с указанием их назначения в эмуляторе.

Таблица 2: Реализованные точки перехвата CUDA Runtime API

Функция	Назначение
<code>__cudaRegisterFatBinary</code>	Регистрация модуля; вызов <code>cuobjdump</code> для извлечения PTX
<code>__cudaRegisterFunction</code>	Отображение дескриптор $\rightarrow$ имя ядра
<code>cudaLaunchKernel</code>	Запуск ядра через диспетчер
<code>cudaMalloc</code>	Выделение памяти устройства
<code>cudaFree</code>	Освобождение памяти устройства
<code>cudaMemcpy</code>	Копирование памяти хост $\leftrightarrow$ устройство
<code>cudaMemset</code>	Инициализация памяти
<code>cudaDeviceSynchronize</code>	Синхронизация (по-оп в эмуляторе)

3.1.3 Извлечение PTX-текста через `cuobjdump`

При перехвате `__cudaRegisterFatBinary(handle)` эмулятор не разбирает fatbinary напрямую. Вместо этого он определяет путь к исполняемому файлу приложения из переменной окружения `CUEMU_TARGET_EXEC` и передаёт его утилите `cuobjdump`:

Листинг 1: Извлечение PTX через `cuobjdump`

```
cuobjdump --dump-ptx <binary> > ptx_text.txt
```

`cuobjdump` извлекает PTX-текст из секции `.nv_fatbin` ELF-файла `[? ]` и выводит его в стандартный вывод. Эмулятор читает этот вывод и передаёт PTX-текст в `PtxModule::Make()`, которая создаёт модуль и строит таблицу инструкций. Полученный `Module` сохраняется в `GlobalContext` под ключом-дескриптором.

Алгоритм извлечения PTX:

1. Прочитать путь к бинарному файлу из `CUEMU_TARGET_EXEC`.
2. Выполнить `cuobjdump -dump-ptx <binary>`, захватить `stdout` через `popen() [? ]`.
3. Передать полученный текст в `PtxModule::Make()`, получить объект `Module` с таблицей инструкций.
4. Сохранить `Module` в `GlobalContext` под ключом-дескриптором.

Формат fatbinary не документирован публично и меняется между версиями CUDA, поэтому разбирать его напрямую нецелесообразно; зависимость от `cuobjdump` оказывается меньшим злом.

3.1.4 Регистрация функций: отображение дескриптор $\rightarrow$ имя ядра

CUDA Runtime API определяет, что при вызове `cudaLaunchKernel` ядро идентифицируется *указателем на функцию-заглушку* в сегменте кода, а не строковым именем. Это обусловлено тем, что в C++ ядра компилируются в обычные функции, и CUDA не гарантирует сохранение символьных имён после компоновки.

При перехвате `__cudaRegisterFunction` эмулятор создаёт отображение:

$$functions_ [ptr] = name, \quad (14)$$

где `ptr` — числовое представление указателя на заглушку, `name` — строка вида `"_Z11vadd_kernelPfS_S_i"`.

При последующем вызове `cudaLaunchKernel(ptr, ...)` эмулятор извлекает строковое имя из отображения по ключу `ptr` и напрямую ищет функцию в `PtxModule::GetEntryFunction(name)` без демангла — PTX-имя уже является декорированным символом, которым `cuobjdump` подписывает каждую точку входа.

Использование указателя вместо строкового имени — не случайное решение: компоновщик C++ не гарантирует сохранение символьных имён, поэтому CUDA идентифицирует ядра по адресу заглушки в сегменте кода. Эмулятор следует той же логике, что позволяет корректно работать даже когда два ядра имеют одинаковые «читаемые» имена в разных пространствах имён.

3.1.5 Запуск cuemu: утилита командной строки и переменные окружения

Утилита командной строки `cuemu` является точкой входа в программу: она подготавливает окружение и повторно запускает целевой бинарный файл:

Листинг 2: Логика запуска `cuemu`

```
export LD_PRELOAD=$CUEMU_LIB_DIR/libemuruntime.so
export LD_LIBRARY_PATH=$CUEMU_LIB_DIR:$LD_LIBRARY_PATH
export CUEMU_TARGET_EXEC="$1"
export CUEMU_COLLECT_PROFILING="{CUEMU_COLLECT_PROFILING:-0}"
export CUEMU_PROFILING_OUTPUT="{CUEMU_PROFILING_OUTPUT:-profiling.txt}"
exec "$@"
```

Поддерживаемые переменные окружения перечислены в таблице 3.

Переменные окружения выбраны как механизм конфигурации намеренно: в системах CI командная строка целевого приложения зафиксирована в конфигурационном файле и не должна меняться при подключении эмулятора.

Таблица 3: Переменные окружения эмулятора

Переменная	Назначение	По умолчанию
CUEMU_COLLECT_PROFILING	Включить сбор метрик	0
CUEMU_PROFILING_OUTPUT	Путь к файлу вывода	profiling.txt
CUEMU_GPU_CONFIG	Путь к TOML-конфигу GPU	встроенный
CUEMU_TARGET_EXEC	Имя целевого бинарного файла	обязателен

4 Синтаксический анализатор РТХ

4.1 Требования к анализатору и выбор стратегии

4.1.1 Анализ структуры РТХ-текста

РТХ-текст имеет строго линейную структуру: последовательность директив, за которыми следуют тела функций. Каждое тело функции — последовательность базовых блоков; каждый базовый блок — последовательность инструкций, предваряемых необязательной меткой. В отличие от языков программирования общего назначения, РТХ не содержит вложенных выражений или скобочных структур внутри отдельных инструкций.

Эти свойства обосновывают отказ от построения полного AST и использование однопроходного линейного разбора. Сложность: $O(N)$ по числу строк N , что критически важно для РТХ-файлов размером 5–50 КБ, возникающих при встраивании трансцендентных функций компилятором NVCC. Применение универсальных инструментов синтаксического анализа (ANTLR, PEG-грамматики) для анализа РТХ избыточно: грамматика РТХ не содержит неоднозначностей, набор мнемоник и директив фиксирован спецификацией ISA 7.x [?], а выполнение в среде без сетевого доступа исключает зависимость от внешних кодогенераторов во время сборки.

4.1.2 Однопроходный инкрементный разбор

Анализатор реализует конечный автомат [?] с тремя состояниями: $Q = \{q_{\text{init}}, q_{\text{func}}, q_{\text{block}}\}$ и следующими переходами:

- $q_{\text{init}} \xrightarrow{\text{.entry/.func}} q_{\text{func}}$: встретили заголовок функции.
- $q_{\text{func}} \xrightarrow{\text{\$label:}} q_{\text{block}}$: встретили метку базового блока.
- $q_{\text{block}} \xrightarrow{\text{instruction}} q_{\text{block}}$: добавляем инструкцию в текущий блок.
- $q_{\text{block}} \xrightarrow{\text{}} q_{\text{init}}$: закрывающая скобка тела функции.

На каждом шаге поддерживается контекст: текущая функция, текущий базовый блок, и монотонный счётчик РС.

Трёх состояний достаточно: РТХ запрещает вложенные определения функций, а базовые блоки внутри тела строго не пересекаются и упорядочены по тексту — в отличие от языков общего назначения с произвольной вложенностью областей видимости, требующей магазинного автомата. Разбор выполняется за $O(N)$ по числу строк без возвратов; результат — объект `PtxModule` с таблицей инструкций, графом базовых блоков и таблицей символов для разрешения переходов.

4.2 Реализация анализатора

4.2.1 Разбор объявлений функций и параметров

Заголовок функции разбирается регулярным выражением `[? ? ? ]` (файл `module.cpp`):

Листинг 3: Регулярное выражение для заголовка функции (`module.cpp`)

```
constexpr std::string_view Pattern =  
    R"((.*)\s\.(entry|func)\s([A-Za-z0-9_+]\s)*\s*\n?\s*{([^\s}]+\s)*})";  
static const std::regex Re(Pattern.data(),  
    std::regex::ECMAScript | std::regex::optimize);
```

Результат — объект `Function` с таблицей параметров `params_` и флагом `isEntry()`, по которому диспетчер различает точки входа ядра и вспомогательные функции. Параметры функции разбираются последовательно: каждый элемент вида `.param .align N .bM name` порождает запись `ParamSlot{offset, size}`, используемую исполнителем при копировании аргументов вызова в регистровый файл.

Директива `.reg .type %prefix<N>` объявляет N регистров заданного типа. Анализатор сохраняет: `reg_decls[func][prefix] = (count, type)`. Эти данные используются исполнителем при инициализации регистрового файла варпа — выделяется ровно столько слотов, сколько объявлено в РТХ, без избыточного выделения. Профилировщик дополнительно использует эту таблицу для подсчёта `write_ops` по префиксу.

4.2.2 Таблица меток и граф базовых блоков

Таблица `BasicBlockAddrMap` является ключевой структурой, связывающей синтаксический анализатор с исполнителем управляющего потока. Метка вида `$BB0_3`: фиксирует точку входа в базовый блок:

1. Закрываем текущий базовый блок, сохраняем его в `PtxModule`.
2. Записываем `basic_blocks["$BB0_3"] = current_pc` — отображение имени метки в монотонный РС первой инструкции блока.
3. Открываем новый базовый блок.

При исполнении инструкции ветвления `bra $L` исполнитель вызывает `GetBasicBlockOffset("$L")` и устанавливает РС варпа в полученное значение. Без этой таблицы реализация переходов потребовала бы линейного поиска по всей таблице инструкций.

4.2.3 Разбор отдельной инструкции

Разбор каждой строки-инструкции выполняется в два последовательных регулярных прохода.

Фаза 1: идентификация мнемоники. Функция `parseInstructions` (файл `function.cpp`) применяет к каждой строке одно регулярное выражение:

Листинг 4: Внешнее регулярное выражение идентификации мнемоники (`function.cpp`)

```
constexpr std::string_view Pattern = "^\\.?(%p[0-9]+\\s)?([a-z]+2?).*$";
```

Первая группа захватывает опциональный предикатный префикс `%pN`, вторая — непосредственно мнемонику (например, `fma`). По имени мнемоники функция `makeInstruction` диспетчеризует управление в соответствующий автогенерированный класс.

Фаза 2: Специальное регулярное выражение. Каждый класс `{name}Instruction` порождается скриптом `autogen.py` из описания в `instructions.json` и компилируется в файл `instructions.cpp`. Статический метод `Make(line)` применяет единственное скомпилированное регулярное выражение, охватывающее всю строку, и одним вызовом `std::regex_match` извлекает все поля — предикат, модификаторы и операнды. Например, для `fma`:

Листинг 5: Фрагмент автогенерированного `instructions.cpp` для `fma`

```
// regex_template (instructions.json):  
//  
"fma\\.($mode)\\.($data)\\s*($dst),\\s*($src1),\\s*($src2)?($imm1)?,\\s*($src3)?($imm2)?;"  
constexpr std::string_view Pattern = R"ptx(...)ptx";  
static std::regex re(Pattern.data(),  
                    std::regex::ECMAScript | std::regex::optimize);  
std::smatch match;  
if (std::regex_match(line, match, re)) {  
    instr->mode_ = FromString<fmamodeQ1>(match[1].str()); // rn/rz/rm/rp  
    instr->data_ = FromString<localFmaDataType>(match[2].str()); // f32/f64  
    instr->dst_ = FromString<registerOperand>(match[3].str());  
    // ... src1, src2, imm1, src3, imm2  
}
```

Каждая группа захвата непосредственно соответствует одному типизированному полю класса; преобразование выполняется функцией `FromString<T>()`. Это исключает ручной разбор строки в горячем цикле исполнителя: все данные уже находятся в типизированных полях к моменту первого вызова `Execute`. Флаг `std::regex::optimize` указывает стандартной библиотеке выполнить дополнительную оптимизацию конечного автомата при первой компиляции регулярного выражения; поскольку объект `re` объявлен статическим, компиляция происходит ровно один раз за время работы процесса вне зависимости от числа разбираемых RTX-файлов.

Присвоение монотонного РС. После успешного создания объекта инструкции счётчик увеличивается на единицу; значение используется как индекс в `instruction_table_` и как ключ в записях `profiling.txt`.

4.2.4 Разрешение операндов

Классификация каждого операнда производится единожды при разборе `[? ]`, а не при каждом исполнении инструкции:

- **Именованный регистр:** `%r5, %f12, %rd0, %p3`. Префикс однозначно определяет тип ($r \rightarrow s32, f \rightarrow f32, rd \rightarrow u64, p \rightarrow pred$) и индекс слота в регистровом файле варпа.
- **Числовой литерал:** десятичный (42), шестнадцатеричный (`0x3f800000`) или IEEE 754 (`0f3f800000`). Значение преобразуется в `uint64_t` при разборе и хранится непосредственно в `Operand`.
- **Адресное выражение:** `[%rd5+8]` — базовый регистр и смещение разделяются при разборе; исполнитель вычисляет адрес как `reg[base] + imm` без дополнительного разбора строки.
- **Символическая метка:** `$BB0_3` — хранится как строка; при первом исполнении перехода разрешается через `BasicBlockAddrMap` и кэшируется.

4.3 Внутреннее представление инструкций

4.3.1 Иерархия классов инструкций и монотонный счётчик PC

Каждая PTX-инструкция представлена отдельным классом, наследующим абстрактный базовый класс:

Листинг 6: Базовый класс (`build/app/ptx_asm/autogen/instructions.h`)

```
class Instruction {
public:
    virtual void Execute(std::shared_ptr<WarpContext>& wc) = 0;
    virtual std::string_view Name() const = 0;
    virtual bool HasExplicitPredicate() const { return false; }
};
```

Конкретные подклассы генерируются из `instructions.json`. Каждый хранит только поля, специфичные для своей мнемоники (пространство памяти, тип данных, режим округления, операнды):

Листинг 7: Пример: класс `ldInstruction`

```
class ldInstruction : public Instruction {
public:
    ldspaceQl space_; // Global, Shared, Local, Const
    dataType data_; // F32, F64, U32, ...
    addressOperand addr_; // base register + immediate offset
    registerOperand dst_; // destination register
    std::optional<registerOperand> pred_; // predicate

    void Execute(std::shared_ptr<WarpContext>& wc) override;
    std::string_view Name() const override { return "ld"; }
    bool HasExplicitPredicate() const override;
};
```

Монотонный PC присваивается инструкции при разборе PTX и хранится в `InstructionList` модуля как индекс в глобальном векторе. При исполнении `Module::GetInstruction(pc)` возвращает `const Instruction&` за $O(1)$ через `instruction_table_[pc]`.

Тот же РС используется как ключ в записях `profiling.txt`, обеспечивая инъективное соответствие между трассой и таблицей инструкций.

5 Система исполнения инструкций

5.1 Таблица диспетчеризации

5.1.1 Структура: фабрика и виртуальный вызов

Каждая RTX-инструкция представлена отдельным C++-классом, наследующим абстрактный базовый класс `Instruction`. Создание объекта нужного подкласса выполняется фабричной функцией

`makeInstruction(name, line)`, которую генерирует `autogen.py` из `instructions.json`:

Листинг 8: Фрагмент автогенерируемой фабрики

```
std::shared_ptr<Instruction> makeInstruction(  
    const std::string& name, const std::string& line) {  
    if (name == "add") return addInstruction::Make(line);  
    if (name == "fma") return fmaInstruction::Make(line);  
    if (name == "ld") return ldInstruction::Make(line);  
    // ... all supported instructions ...  
}
```

Фабрика вызывается один раз при разборе RTX-текста. В горячем цикле исполнения варпа производится единственный виртуальный вызов: `instr->Execute(warp_ctx)`. Накладные расходы — одна косвенная адресация через таблицу виртуальных методов (`vtable`).

Фаза создания объектов (разбор) и фаза исполнения (горячий цикл) разделены: анализатор проверяет синтаксис, исполнитель реализует семантику.

5.1.2 Генерация кода классов инструкций

Скрипт `autogen.py` читает `instructions.json` и генерирует файлы `instructions.h/.cpp` с классом для каждой мнемоники.

Принцип открытости/закрытости: добавление поддержки новой инструкции сводится к добавлению записи в `instructions.json` и реализации метода `Execute()` — `autogen.py` вставит нужную ветку в фабрику автоматически.

Аналогичный принцип применяется в системе метрик профилировщика (Глава 7): единый JSON-файл управляет генерацией кода сразу нескольких компонентов, что исключает рассинхронизацию.

5.2 Регистровый файл и разрешение операндов

5.2.1 Структура регистрового файла

Регистровый файл варпа организован как вектор на поток, где для каждого потока хранится отображение типа регистра на вектор значений:

Листинг 9: Структура регистрового файла

```
using RegisterContext = std::vector<uint64_t>;
using ThreadContext = std::unordered_map<Ptx::registerType,
                                         RegisterContext>;
// thread_regs[thread_id][registerType][slot]
std::vector<ThreadContext> thread_regs;
```

Ключ `registerType` — перечисление `{R, Rd, F, Fd, P, ...}`; вектор значений индексируется номером регистра из PTX-операнда. Все значения хранятся в `uint64_t`: покрывает `f64` без потерь; значения `f32` занимают младшие 32 бита. Преобразование выполняется через `std::bit_cast` (C++20).

Размер внутренних векторов определяется директивами `.reg` в PTX: исполнитель выделяет ровно столько слотов, сколько объявлено, без избыточного выделения памяти.

5.2.2 Вспомогательные шаблоны `reg_cast<T>` и `to_u64<T>`

Все исполнители инструкций работают с регистровым файлом через два шаблонных примитива, определённых в `executors.hpp`:

Листинг 10: Примитивы доступа к регистрам

```
template<typename T>
T reg_cast(uint64_t raw) {
    if constexpr (std::is_integral_v<T>)
        return static_cast<T>(raw);
    else {
        T val;
        std::memcpy(&val, &raw, sizeof(T));
        return val;
    }
}

template<typename T>
uint64_t to_u64(T val) {
    if constexpr (std::is_integral_v<T>)
        return static_cast<uint64_t>(
            static_cast<std::make_unsigned_t<T>>(val));
    else {
        uint64_t raw = 0;
        std::memcpy(&raw, &val, sizeof(T));
        return raw;
    }
}
```


Целые числа преобразуются посредством `static_cast` через беззнаковый промежуточный тип — бит-паттерн знаковых значений в `uint64_t` сохраняется. Числа с плавающей запятой сериализуются через `memcpu`.

Хранение всех значений в `uint64_t` исключает шаблонный параметр по типу из структуры `ThreadContext` и сокращает объём кода регистрового файла без потери точности.

5.2.3 Вычисление эффективного адреса

Эффективный адрес потока t для операнда `[%rd5+8]`:

$$ea(t) = R[t][u64][5] + 8, \quad (15)$$

где `imm_offset = 8` извлекается синтаксическим анализатором из адресного выражения. Вычисленный адрес $ea(t)$ передаётся в делегирующий метод `WarpContext`, который маршрутизирует обращение в нужное пространство памяти на основе функции $space(ea(t))$.

5.3 Исполнители арифметических инструкций

5.3.1 Целочисленная арифметика

Поведение ключевых целочисленных инструкций:

`add.u32 %rd, %ra, %rb:`

$$R'[t][u32][d] = (R[t][u32][a] + R[t][u32][b]) \bmod 2^{32}, \quad \forall t \in \text{active}(M). \quad (16)$$

`mul.hi.u32 %rd, %ra, %rb:`

$$R'[t][u32][d] = \left\lfloor \frac{R[t][u32][a] \cdot R[t][u32][b]}{2^{32}} \right\rfloor, \quad \forall t \in \text{active}(M). \quad (17)$$

`div.s32 %rd, %ra, %rb:`

$$R'[t][s32][d] = \text{trunc} \left(\frac{R[t][s32][a]}{R[t][s32][b]} \right), \quad \forall t \in \text{active}(M), R[t][s32][b] \neq 0. \quad (18)$$

Граничные случаи: деление на ноль — по спецификации РТХ возвращает 0; переполнение `INT_MIN / -1` — возвращает `INT_MIN` (неопределённое поведение в стандарте C++, обрабатывается явно).

Реальный GPU воспроизводит эти граничные случаи аппаратно согласно РТХ-спецификации, поэтому эмулятор обрабатывает их явно — иначе результаты расходились бы при корректном поведении на типичных путях.

Таблица 4: Режимы округления PTX и их соответствие IEEE 754

Суффикс PTX	Режим IEEE 754	Описание
.rn	roundTiesToEven	к ближайшему чётному
.rz	roundTowardZero	усечение
.ru	roundTowardPositive	к $+\infty$
.rd	roundTowardNegative	к $-\infty$

5.3.2 Арифметика с плавающей запятой: IEEE 754 и режимы округления

PTX определяет четыре модификатора округления, соответствующие режимам IEEE 754-2008 [?]:

Модификаторы округления (.rn, .rz, .ru, .rd) разбираются синтаксическим анализатором и сохраняются в объекте инструкции. В текущей реализации эмулятор не переключает режим округления хост-CPU через fesetround: операции с плавающей запятой выполняются с режимом FE_TONEAREST по умолчанию. Это известное ограничение, которое не влияет на корректность результата для инструкций, не чувствительных к последнему биту (*faithful rounding*).

5.3.3 Инструкции преобразования типов

cvt.rni.s32.f32 %rd, %fa: преобразование f32 $\rightarrow$ s32 с округлением к ближайшему целому:

$$R'[t][s32][d] = \text{round}_n(R[t][f32][a]), \quad \forall t \in \text{active}(M), \quad (19)$$

где round_n — округление к ближайшему чётному (roundTiesToEven по IEEE 754).

cvt.rz.f32.s64 %fd, %ra: преобразование s64 $\rightarrow$ f32 с усечением дробной части:

$$R'[t][f32][d] = \text{round}_z(R[t][s64][a]), \quad \forall t \in \text{active}(M). \quad (20)$$

5.4 Исполнители инструкций работы с памятью

5.4.1 ld.global / ld.shared: чтение

Исполнитель ld реализован как шаблонный метод `ldInstruction::ExecuteThread<dataType Data>`, параметризованный типом данных. Тип `ptx_native_t<Data>` — псевдоним, отображающий перечислитель `dataType` на соответствующий C++-тип (F32 $\rightarrow$ float, S64 $\rightarrow$ int64_t и т.д.):

Листинг 11: `ldInstruction::ExecuteThread` (executors.tpp)

```
template<dataType Data>
void ldInstruction::ExecuteThread(
    uint32_t lid, std::shared_ptr<WarpContext>& wc)
{
    using T = ptx_native_t<Data>;
```

```

uintptr_t addr = 0;
if (addr_.reg.type != registerType::UNDEFINED) {
    addr = static_cast<uintptr_t>(
        wc->thread_regs[lid][addr_.reg.type][addr_.reg.reg_id]);
    addr += static_cast<ptrdiff_t>(addr_.imm);
}
if (space_ == ldspaceQ1::Shared) {
    auto* base = static_cast<uint8_t*>(wc->getSharedBase());
    addr = reinterpret_cast<uintptr_t>(base + addr);
}
T val = T{};
std::memcpy(&val, reinterpret_cast<const void*>(addr), sizeof(T));
wc->thread_regs[lid][dst_.type][dst_.reg_id] = to_u64<T>(val);
}

```

Метод вызывается по одному разу для каждого активного потока варпа. Переключение между пространствами памяти (`global/shared`) производится внутри метода по полю `space_`, разобранным синтаксическим анализатором. Данные копируются через `memcpy`.

5.4.2 `st.global / st.shared`: запись с учётом маски

Для каждого потока $t \in \text{active}(M)$: $*ptr(ea(t)) \leftarrow R[t][\tau][src]$. Поток с $M[i] = 0$ пропускают запись — это семантически обязательно: неактивные потоки не должны изменять состояние памяти даже при вычислении адреса.

5.4.3 `setp / selp`: операции с предикатами

Для `setp.lt.f32 %p, %fa, %fb`:

$$P'[t] = (R[t][f32][a] < R[t][f32][b]), \quad \forall t \in \text{active}(M). \quad (21)$$

Результат сравнения (`true/false`) записывается в один предикатный регистр `dst_` как значение 1/0 типа `uint64_t`.

5.5 Исполнители инструкций управления потоком

5.5.1 Безусловный переход `bra.uni $L`

$$\langle \text{bra.uni } \$L, \sigma \rangle \rightarrow \sigma[\text{PC} \leftarrow \text{offset}(\$L)]. \quad (22)$$

Суффикс `.uni` (`uniform`) гарантирует конвергентность. В реализации: `warp.pc = warp.GetBasicBlockPc("$L")`. Стек дивергенции не модифицируется.

5.5.2 Условный переход @%p bra \$L

Разбор по случаям (подробно в Главе 6):

$$M_{\text{taken}} = M \wedge P, M_{\text{not}} = M \wedge \neg P.$$

- **Случай 1** ($M_{\text{not}} = 0$): $PC' = \text{offset}(\$L)$.
- **Случай 2** ($M_{\text{taken}} = 0$): $PC' = PC + 1$.
- **Случай 3**: Вызов `PDOM_Diverge` (см. Главу 6).

Инструкция `ret` завершает исполнение варпа: она сбрасывает устаревшие фреймы PDOM-стека дивергенции и устанавливает `pc = EOC` (End-of-Code), сигнализируя циклу исполнения об окончании:

Листинг 12: `retInstruction::ExecuteBranch` (`executors.tpp`)

```
// Discard stale convergence frames from guard-clause paths.
while (!wc->execution_stack.empty()
      && wc->execution_stack.top().is_convergence)
    wc->execution_stack.pop();

if (wc->execution_stack.empty())
    wc->pc = WarpContext::EOC;
else {
    const auto frame = wc->execution_stack.top();
    wc->execution_stack.pop();
    wc->pc = frame.pc;
    wc->execution_mask = frame.mask;
}
```

5.5.3 `bar.sync`: делегирование барьерной синхронизации

Исполнитель `bar.sync` N вызывает `warp.syncBarrier()`, реализованный в компонентах управления блоком (диспетчер блока (`rt_stream.cpp`)).

Математически: все записи `st.shared` до `bar.sync` гарантированно видимы всем потокам после него. Гергель и Стронгин [?, с. 142] формулируют это требование так: «изменение каким-либо потоком значений общих переменных должно приводить к блокировке доступа к модифицируемым данным для всех остальных потоков». Барьер обеспечивает именно это свойство — полную видимость памяти (`happens-before` по отношению к записям разделяемой памяти).

Реализация барьера в эмуляторе зависит от режима сборки. При компиляции с поддержкой OpenMP [?] (`EMULATOR_OPENMP_ENABLED`) каждый варп блока выполняется в отдельном потоке ОС. Технология OpenMP [?, с. 141] использует директивы параллелизма «для выделения в программе параллельных областей (`parallel regions`), в которых последовательный исполняемый код может быть разделён на несколько отдельных командных потоков (`threads`)». В реализации эмулятора это достигается директивой `#pragma omp`

`parallel for; bar.sync` реализован через `BlockBarrier` — циклический барьер на базе `std::mutex` и `std::condition_variable`. Варп, достигший барьера, блокируется до тех пор, пока все активные варпы блока не придут к той же точке; последний варп вызывает `WarpDone()`, снимая блокировку остальных.

При сборке без OpenMP применяется кооперативный планировщик (*round-robin*): на каждой итерации каждый варп исполняет инструкции до ближайшего `bar.sync` включительно, после чего управление передаётся следующему варпу. Когда все варпы достигли барьера, они продолжают исполнение совместно. Эта схема корректно воспроизводит семантику барьерной синхронизации без параллелизма на уровне ОС.

6 Механизм управления дивергентным потоком

6.1 Теория PDOM-реконвергенции

6.1.1 Граф управляющего потока и постдоминаторы

Граф управляющего потока (CFG) программы определяется как:

$$G = (V, E, \text{entry}, \text{exit}), \quad (23)$$

где V — множество базовых блоков, $E \subseteq V \times V$ — рёбра передачи управления, *entry* и *exit* — специальные узлы.

Определение (постдоминатор) [? ?]. Узел D является постдоминатором узла N ($D \text{ pdom } N$), если каждый путь из N в *exit* проходит через D .

Определение (непосредственный постдоминатор). Непосредственный постдоминатор узла N :

$$\text{IPDOM}(N) = \underset{D: D \text{ pdom } N, D \neq N}{\operatorname{argmin}} |E^*[N, D]|, \quad (24)$$

то есть ближайший постдоминатор (за минимальное число шагов).

Теорема [?]. $\text{IPDOM}(N)$ существует для каждого узла $N \neq \text{exit}$.

Доказательство строится по индукции по структуре CFG: сначала для ациклических CFG, затем обобщается на произвольные CFG методом фиксированной точки [?].

Постдоминатор гарантирует, что все пути из N к *exit* проходят через $\text{IPDOM}(N)$ — именно в этой точке разошедшиеся потоки варпа обязаны встретиться вновь.

6.1.2 Теорема о корректности PDOM-реконвергенции

Теорема (PDOM-корректность) [? ?]. Пусть при исполнении условного перехода N варьируется активность потоков: $M_{\text{taken}} \neq 0$ и $M_{\text{not}} \neq 0$. Если точкой реконвергенции выбирается $\text{IPDOM}(N)$, то:

1. Все потоки варпа воссоединятся в $\text{IPDOM}(N)$.
2. Ни один поток не пропустит инструкцию своего активного пути.
3. После реконвергенции маска M восстанавливается в полное значение.

Обоснование.

Утверждение 1 (воссоединение). Введём обозначения: *ветка перехода* — подмножество потоков с $P = 1$, которые переходят на метку $\$L$; *ветка продолжения* — подмножество потоков с $P = 0$, которые продолжают выполнение с $\text{PC} + 1$. По определению постдоминатора каждый путь из N в *exit* проходит через $\text{IPDOM}(N)$. Ветка перехода ($N \rightarrow \$L \rightarrow \dots$) и ветка продолжения ($N \rightarrow \text{PC} + 1 \rightarrow \dots$) — оба пути ведут в *exit*, следовательно, оба обязательно пройдут через $\text{IPDOM}(N)$. Значит, все потоки встретятся в этой точке.

Утверждение 2 (полнота путей). Потоки с маской M_{taken} исполняются только внутри ветки перехода, а потоки с маской M_{not} — только внутри ветки продолжения. Ни один поток не получает нулевую маску пока его ветка активна, поэтому ни одна инструкция активного пути не пропускается.

Утверждение 3 (восстановление маски). Когда ветка перехода завершается безусловным прыжком вперёд, алгоритм `PDOM_Merge` вычисляет полную маску $M = M_{\text{taken}} \vee M_{\text{not}}$ и помещает в стек фрейм реконвергенции $\{\text{IPDOM}(N), M, \text{true}\}$. Достигнув адреса $\text{IPDOM}(N)$, диспетчер снимает этот фрейм и восстанавливает полную маску — варп снова исполняется единым потоком.

Поскольку выбирается *непосредственный* постдоминатор — ближайший к N — последовательное исполнение двух ветвей минимально: потоки воссоединяются как можно раньше.

Выбор *непосредственного* постдоминатора также влияет на метрику `branch_efficiency`: чем раньше маска восстанавливается, тем меньше инструкций выполняется с частичной загрузкой варпа.

6.2 Структура данных стека дивергенции

6.2.1 Тип `StackFrame` и семантика полей

Каждый фрейм стека дивергенции описывается структурой:

Листинг 13: Структура `StackFrame`

```
struct StackFrame {
    uint64_t pc;
    uint32_t mask;
    bool     is_convergence;
};
```

Инварианты:

- Если `is_convergence = true`: при достижении $\text{PC} = \text{frame.pc}$ маска восстанавливается в `frame.mask` и фрейм снимается.
- Если `is_convergence = false`: дивергентный путь в очереди; при достижении его РС исполнение возобновляется с маской `frame.mask`.

Стек $S = [f_n, \dots, f_1, f_0]$ (вершина — f_n): фрейм реконвергенции всегда располагается *ниже* соответствующих дивергентных фреймов.

6.2.2 Сложность операций и ограничение глубины

Операции `push/pop/top` — $O(1)$. При вложенности дивергенций глубиной k : $|S| \leq 2k$.

Для типичных CUDA-ядер $k \leq 5 - |S| \leq 10$. Объём памяти: $\text{sizeof}(\text{StackFrame}) \cdot 10 = 160$ байт.

Ограниченность глубины стека — следствие архитектурных ограничений реальных GPU: аппаратный стек дивергенции также имеет фиксированный размер, что вынуждает компилятор nvcc ограничивать глубину вложенных условных конструкций.

6.3 Алгоритм обработки условного перехода

6.3.1 Алгоритм PDOM_Diverge

Алгоритм 1: PDOM_Diverge($M, P, \$L$) — условный переход `@pN bra $L [? ]`

Input: маска исполнения M , предикатная маска P , метка перехода $\$L$

Output: обновлённые M , PC, стек S

```

1  $M_{\text{taken}} \leftarrow M \wedge P$ ;
2  $M_{\text{not}} \leftarrow M \wedge \neg P$ ;
3 if  $M_{\text{taken}} = 0$  then PC  $\leftarrow$  PC + 1; return;
4 if  $M_{\text{not}} = 0$  then PC  $\leftarrow$  offset( $\$L$ ); return;
  // дивергенция: отложить ветку продолжения, исполнить ветку перехода
5 push( $S, \{\text{PC} + 1, M_{\text{not}}, \text{is\_conv} = \text{false}\}$ );
6  $M \leftarrow M_{\text{taken}}$ ;
7 PC  $\leftarrow$  offset( $\$L$ );
```

Пояснение к шагам. Точка реконвергенции (IPDOM) на момент условного перехода неизвестна: NVCC не кодирует её в инструкции. Шаг 5 откладывает ветку продолжения как фрейм ожидания $\{\text{PC} + 1, M_{\text{not}}, \text{false}\}$. Шаги 6–7 немедленно переходят к ветке перехода с маской M_{taken} . IPDOM обнаруживается позже, когда ветка перехода заканчивается безусловным прыжком вперёд (алгоритм PDOM_Merge ниже).

Выбор приоритета в пользу ветки перехода (M_{taken}) перед веткой продолжения является соглашением реализации, а не требованием спецификации PTX. Аналогичный порядок применяется в оригинальном аппаратном стеке PDOM архитектур до Volta [? ?]: сначала исполняется взятый переход, затем — ветка fall-through.

6.3.2 Обнаружение IPDOM и цикл реконвергенции

Обнаружение точки слияния (PDOM_Merge). NVCC всегда завершает ветку перехода безусловным прыжком вперёд к точке слияния. Исполнитель `braInstruction::ExecuteBranch` обрабатывает этот прыжок следующим образом:

Алгоритм 2: PDOM_Merge — безусловный bra \$L при непустом стеке

```
1  $t \leftarrow target\_pc$ ;  
2 if not  $S.empty()$  and  $t > PC$  then  
3   if  $\neg S.top().is\_conv$  and  $t > S.top().pc$  then  
4     // Case A: ветка перехода завершена;  $t$  -- IPDOM  
5      $M_{ft} \leftarrow S.top().mask$ ;  $pc\_ft \leftarrow S.top().pc$ ;  
6      $M_{full} \leftarrow M \vee M_{ft}$ ;  
7      $S.pop()$ ;  
8     push( $S, \{t, M_{full}, is\_conv = true\}$ );  
9      $M \leftarrow M_{ft}$ ;  $PC \leftarrow pc\_ft$ ;  
10    return;  
11  else  
12    if  $S.top().is\_conv$  and  $S.top().pc = t$  then  
13      // Case B: ветка продолжения пришла к IPDOM  
14       $M \leftarrow S.top().mask$ ;  $S.pop()$ ;  
15       $PC \leftarrow t$ ; return;  
16  $PC \leftarrow t$ ; // обычный безусловный прыжок
```

Встроенный цикл реконвергенции. Перед каждой инструкцией диспетчер (`rt_stream.cpp`) выполняет цикл, который снимает фреймы реконвергенции (`is_conv = true`), чей PC совпадает с текущим:

Алгоритм 3: Встроенный цикл реконвергенции (`rt_stream.cpp`)

```
1 while not  $S.empty()$  and  $S.top().is\_conv$  and  $S.top().pc = PC$  do  
2    $M \leftarrow S.top().mask$ ;  
3    $S.pop()$ ;
```

Фреймы ожидания (`is_conv = false`) цикл не трогает: они активируются только внутри PDOM_Merge (Case A). Сложность: $O(k)$ за одну проверку, $O(k \cdot N_{instructs})$ суммарно для ядра с k уровнями дивергенции.

6.4 Граничные случаи

6.4.1 Вложенные дивергенции: корректность при $k \geq 2$

Рассмотрим пример: внешний дивергентный if и внутренний дивергентный if. Пусть начальная маска $M_0 = 0xFFFFFFFF$, внешний переход на `pcB`, внутренний — на `pcC`:

- Внешний if (`pcB`): алгоритм PDOM_Diverge помещает в стек $f_1 = \{pc_B+1, M_{not_out}, false\}$.
Стек: $[f_1]$. Исполняется ветка перехода с маской M_{taken_out} .
- Внутренний if (`pcC`): алгоритм PDOM_Diverge помещает в стек $f_2 = \{pc_C+1, M_{not_in}, false\}$.
Стек: $[f_1, f_2]$. Исполняется внутренняя ветка перехода.
- Конец внутренней взятой ветки (безусловный bra к `pc_ipdom_in`): PDOM_Merge Case A снимает f_2 , помещает в стек $f_3 = \{pc_ipdom_in, M_{taken_out}, true\}$. Стек: $[f_1, f_3]$.
PC переходит к `pcC+1`.

4. Достижение `pc_ipdom_in`: цикл реконвергенции снимает f_3 , восстанавливает $M = M_{\text{taken_out}}$. Стек: $[f_1]$.
5. Конец внешней взятой ветки (безусловный `bra` к `pc_ipdom_out`): `PDOM_Merge Case A` снимает f_1 , помещает в стек $f_4 = \{\text{pc_ipdom_out}, M_0, \text{true}\}$. Стек: $[f_4]$. PC переходит к `pcB+1`.
6. Достижение `pc_ipdom_out`: цикл реконвергенции снимает f_4 , восстанавливает $M = M_0$. Стек: $[]$.

Порядок LIFO при снятии фреймов гарантирует восстановление масок от внутреннего уровня к внешнему; каждый уровень вложенности добавляет ровно два фрейма в стек вне зависимости от глубины.

6.4.2 Дивергентные циклы

В `while`-цикле с дивергентным условием выхода: часть потоков достигает `bra $exit` раньше остальных. При каждой итерации `PDOM_Diverge` корректно разделяет «продолжающие» (M_{continue}) и «вышедшие» (M_{exit}) потоки. Потоки, завершившие цикл, не участвуют в последующих итерациях, так как их биты сброшены в M_{continue} .

6.4.3 Инструкция `exit` и обнуление маски активных потоков

Инструкция `exit` при $M \neq 0$ обнуляет биты завершивших потоков в маске: $M' = M \setminus \{t \mid \text{поток } t \text{ исполняет } \text{exit}\}$.

При $M' = 0$ метод `isActive()` возвращает `false`: диспетчер блока (`rt_stream.cpp`) останавливает варп. Модуль исполнения варпа только обновляет маску — сигнал останова передаётся через публичный метод.

7 Система профилирования

7.1 Архитектура профилировщика

7.1.1 Хуки профилирования с автогенерацией кода

В основе системы профилирования лежит принцип разделения [?]: метаданные метрик хранятся в JSON-файле `profiling_metrics.json`, а код реализации генерируется скриптом `autogen.py`.

Набор метрик меняется чаще, чем механизм их сбора, поэтому метаданные вынесены в JSON-файл: добавление новой метрики не требует правки C++.

Листинг 14: Фрагмент `profiling_metrics.json`

```
{
  "instructions": {
    "*": { "metrics": ["branch_efficiency"] },
    "ld": { "metrics": ["branch_efficiency",
                      "bank_conflicts",
                      "global_mem_transactions",
                      "global_coalescing"] },
    "st": { "metrics": ["branch_efficiency",
                      "global_mem_transactions",
                      "global_coalescing"] }
  }
}
```

Ключ "*" задаёт метрики для всех инструкций; специализированные ключи добавляют метрики для конкретных мнемоник. Чтобы добавить новую метрику, достаточно дополнить JSON-файл и перезапустить `autogen.py`.

Привязка метрик к конкретным мнемоникам (например, `bank_conflicts` только для `ld`) исключает избыточные вычисления: конфликты банков неприменимы к арифметическим инструкциям и только увеличивали бы объём собираемых данных.

Файл `profilers.hpp` содержит `inline`-функции вычисления метрик и подключается один раз в начале `instructions.cpp`. Для каждой инструкции `autogen.py` генерирует два метода: `Execute()` и `Profile()`. Вызов `Profile()` в конце `Execute()` защищён проверкой с атрибутом `[[unlikely]]`:

Листинг 15: Автогенерируемые `Execute()` и `Profile()` для `ld`

```
void ldInstruction::Execute(std::shared_ptr<WarpContext>& wc) {
    // dispatch ExecuteThread<Data> for each active lane
    if (wc->profiling_buf) [[unlikely]]
        Profile(wc, wc->pc, wc->execution_mask, *wc->profiling_buf);
    wc->pc++;
}

void ldInstruction::Profile(std::shared_ptr<WarpContext>& wc,
    uint64_t pc, uint32_t orig_mask, WarpProfilingBuffer& buf) {
```

```

ProfilingRecord rec;
rec.pc = pc; rec.instr_name = "ld";
rec.metrics.emplace_back("branch_efficiency",
    FormatFloat(ComputeBranchEfficiency(*this, orig_mask, wc)));
rec.metrics.emplace_back("bank_conflicts",
    std::to_string(ComputeBankConflicts(*this, orig_mask, wc)));
rec.metrics.emplace_back("global_coalescing",
    FormatFloat(ComputeGlobalCoalescing(*this, orig_mask, wc)));
buf.push_back(std::move(rec));
}

```

Атрибут `[[unlikely]]` устраняет накладные расходы на горячем пути: при выключенном профилировании предсказатель ветвлений считает эту ветвь маловероятной, и вызов `Profile()` не производится.

7.1.2 Структура `ProfilingRecord` и буферизация

Каждое исполнение инструкции порождает одну запись:

Листинг 16: Структура `ProfilingRecord`

```

struct ProfilingRecord {
    uint64_t    timestamp_ns;
    uint64_t    pc;
    uint32_t    block_id;
    uint32_t    warp_id;
    uint32_t    execution_mask;
    std::string function_name;
    std::string basic_block;
    std::string instr_name;
    std::vector<std::pair<std::string, std::string>> metrics;
};

```

Каждый варп накапливает записи локально в `WarpProfilingBuffer` без синхронизации. Сброс в общий `Profiler` с захватом мьютекса выполняется единожды по завершении блока. Число синхронизаций: $N_{\text{sync}} = N_{\text{blocks}} \ll N_{\text{records}}$, что минимизирует накладные расходы.

7.1.3 Метка времени: `steady_clock`

Листинг 17: Получение временной метки

```

auto ts = std::chrono::steady_clock::now();
uint64_t timestamp_ns = std::chrono::duration_cast<
    std::chrono::nanoseconds>(ts - epoch_).count();

```

Использование `steady_clock` (монотонные часы) вместо `system_clock` гарантирует монотонное возрастание меток времени независимо от корректировок NTP.

7.2 Формальное определения метрик производительности

7.2.1 branch_efficiency: эффективность ветвлений

Пусть M_k — маска исполнения варпа на k -м исполнении инструкции, W — размер варпа (32). Для инструкций без явного предиката доля активных потоков:

$$\eta_k = \frac{\text{popcount}(M_k)}{W}. \quad (25)$$

Для инструкций с явным предикатом `@%p` дополнительно учитывается расхождение потоков на пути «выполнить» / «пропустить» внутри активной маски:

$$\eta_k = \frac{\max(\text{popcount}(M_k \wedge P), \text{popcount}(M_k \wedge \neg P))}{\text{popcount}(M_k)}. \quad (26)$$

Среднее по всем запускам ядра:

$$\bar{\eta} = \frac{1}{K} \sum_{k=1}^K \eta_k, \quad K = \text{issue_count}. \quad (27)$$

7.2.2 bank_conflicts: конфликты банков разделяемой памяти

Разделяемая память организована в $B = 32$ банка шириной $w = 4$ байта [? ?]. Номер банка для адреса a :

$$\text{bank}(a) = \left\lfloor \frac{a}{w} \right\rfloor \bmod B. \quad (28)$$

Число конфликтов при одном исполнении `ld.shared/st.shared`:

$$C = \max_{b \in [0, B)} \left| \{t \in \text{active}(M) \mid \text{bank}(ea(t)) = b\} \right| - 1. \quad (29)$$

$C = 0$ — нет конфликтов (все потоки обращаются к разным банкам); $C = k$ — k -кратный конфликт. При фиксированных $W=32$, $B=32$ сложность вычисления — $O(1)$ на одно исполнение инструкции.

7.2.3 global_coalescing: коэффициент слияния транзакций

Кэш-строка глобальной памяти занимает 128 байт. Число фактических транзакций при одном исполнении:

$$T_{\text{actual}} = \left| \left\{ \left\lfloor \frac{ea(t)}{128} \right\rfloor \mid t \in \text{active}(M) \right\} \right|. \quad (30)$$

Теоретический минимум транзакций при последовательном расположении $|\text{active}(M)|$ элементов размером s байт:

$$T_{\text{ideal}} = \left\lceil \frac{|\text{active}(M)| \cdot s}{128} \right\rceil. \quad (31)$$

Коэффициент слияния:

$$\kappa = \min\left(1, \frac{T_{\text{ideal}}}{T_{\text{actual}}}\right) \in (0, 1]. \quad (32)$$

$\kappa = 1$ соответствует идеально слитному доступу; $\kappa \ll 1$ — сильно разрозненному (stride-доступ, scatter/gather) [?].

7.2.4 write_ops: давление на регистровый файл

Для каждого типа регистров (префикс `%r`, `%f`, `%rd`, ...) накапливается суммарное число записей:

$$W[\tau] = \sum_{\substack{k: \text{инструкция } k \\ \text{пишет в регистр типа } \tau}} \text{popcount}(M_k). \quad (33)$$

Инструкции без регистра назначения (`st`, `bar`, `bra`, `ret`) в сумму не включаются.

7.3 Инструмент генерации отчётов

7.3.1 Анализатор текстового вывода (parser.py)

Файл `profiling.txt` имеет иерархическую структуру: «запуск ядра $\rightarrow$ блок $\rightarrow$ варп $\rightarrow$ инструкция». Анализатор реализует конечный автомат:

1. `await_launch`: ожидание строки `"[LAUNCH] kernel_name grid(...) block(...)"`.
2. `await_block`: ожидание строки `"[BLOCK] block_id=..."`.
3. `in_block`: разбор строк записей регулярным выражением `_RECORD_RE` за $O(1)$ на строку.

На выходе — `list[ProfilingRecord]`, передаваемый в агрегатор.

7.3.2 PTX-анализатор для отчёта (ptx_parser.py)

Python-реализация анализатора решает задачу построения индекса:

$$by_func_pc : (fn, pc) \rightarrow PtxInstr, \quad (34)$$

обеспечивающего $O(1)$ -соответствие между профилировочной записью и PTX-строкой для аннотации исходного кода.

7.3.3 Агрегатор (aggregator.py): формулы накопления

Для каждого уникального РС:

$$exec_count += \text{popcount}(M_k), \quad (35)$$

$$issue_count += 1, \quad (36)$$

$$be_sum += \eta_k, \quad (37)$$

$$bc_total += C_{\text{bank},k}, \quad (38)$$

$$kappa_sum += \kappa_k. \quad (39)$$

Средние значения:

$$\bar{\eta}_{\text{branch}} = \frac{be_sum}{issue_count}, \quad \bar{\kappa} = \frac{kappa_sum}{issue_count}. \quad (40)$$

7.3.4 Рендерер HTML-отчёта (renderer.py)

Самодостаточный HTML-файл содержит пять секций:

1. **Панель «пилюль»:** сводные метрики ядра — число инструкций, потоков, среднее $\bar{\eta}_{\text{branch}}$, суммарные банковские конфликты, средний $\bar{\kappa}$.
2. **Таблица инструкций:** все уникальные РС с тепловой подсветкой по `exec_count`.
3. **Горячие точки:** топ-10 инструкций по дивергенции, банковским конфликтам, коэффициенту слияния.
4. **Таблица регистров:** для каждого префикса — тип, число объявленных слотов, накопленный `write_ops`.
5. **Аннотированный исходный код:** строки `.cu`-файла с метриками инструкций, привязанными через директиву `.loc`.

Все CSS и JS встроены в HTML — один файл без внешних зависимостей.

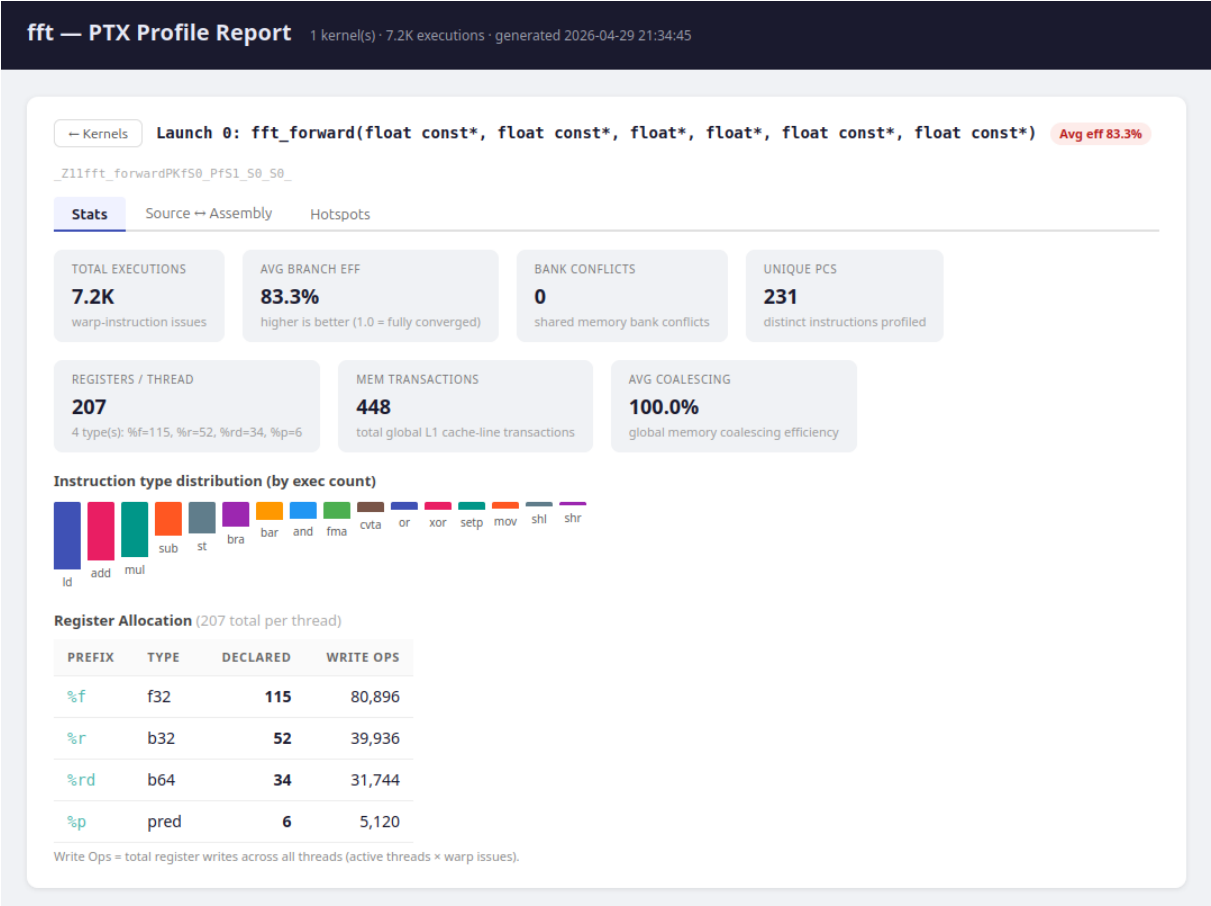


Рисунок 3: HTML-отчёт профилировщика: панель сводных метрик для ядра `fft_custom`.

8 Тестирование и верификация

8.1 Модульные тесты

Для верификации изолированных компонентов эмулятора разработан набор модульных тестов на базе фреймворка GoogleTest [? ?]. Тесты организованы в четыре файла: `test_executors.cpp` (исполнители инструкций), `test_barrier.cpp` (барьерная синхронизация), `test_coalescing.cpp` (объединение транзакций памяти) и `test_ptx_asm.cpp` (разбор PTX-ассемблера). Каждый файл проверяет ровно один подмодуль.

Покрытие кода измеряется инструментом `lcov`.

Покрытие строк составляет **80.4%**, покрытие ветвей — **78.7%**. Непокрытые ветви сосредоточены преимущественно в обработчиках ошибочных состояний (`global_context`, путь выделения GPU-памяти), которые требуют наличия CUDA Runtime и не тестируются модульными тестами. Модульные тесты покрывают изолированные компоненты и служат страховочной сеткой при доработке кода, однако не заменяют интеграционное тестирование на реальных ядрах.

Достигнутый уровень покрытия — свыше 80% строк и ветвей — позволяет верифицировать основной функциональный путь, включающий все классы инструкций и ключевые граничные случаи, описанные в Главе 5. Для производственного инструмента это значение следует повышать, в первую очередь за счёт добавления тестов, имитирующих ошибочные сценарии инициализации контекста.

8.2 Часть I — Интеграционное тестирование CUDA-ядер

Методология. Каждый интеграционный тест компилируется с `nvcc` [?] и запускается под эмулятором: `cuemu ./binary`. Тест самостоятельно сравнивает результат с CPU-эталоном, возвращая код возврата 0 при успехе. Для оценки производительности каждое ядро запускается несколько раз; конкретное число повторений указано в описании каждого ядра. Фиксируются медиана и стандартное отклонение времени `KernelLaunch`. Тесты запускаются на машине без GPU (процессор AMD Ryzen 7 3700U, 8 ГБ RAM).

Семь ядер выбраны таким образом, чтобы перекрывать качественно различные вычислительные паттерны: от простейших поэлементных операций (`vadd`) через ядра с нетривиальным управлением потоком и разделяемой памятью (`softmax`, `mmul`) до сложных алгоритмов со вложенными барьерами и динамическим числом итераций (`attention`, `conj_grad`). Все 7 тестов пройдены без ошибок корректности.

8.2.1 `vadd_custom` — поэлементное сложение векторов

Описание теста. Ядро складывает два массива `float` длиной $N = 1024$ на одном блоке из 32 потоков. CPU-эталон вычисляется скалярным циклом; допуск — абсолютная погрешность $|gpu[i] - ref[i]| \leq 10^{-6}$. Тест проверяет базовый путь: загрузка из глобальной памяти (`ld.global.f32`), арифметику (`add.f32`), запись в глобальную память (`st.global.f32`).

Тестирование. Фиксируются: код возврата (0 = PASS), максимальная абсолютная ошибка, число активных потоков на инструкцию (ожидается ровно 32 — дивергенции нет).

Замеры производительности. Ядро содержит минимальное число инструкций (3 на поток), что делает его базовой точкой отсчёта. Замер проводился методом «50 повторений одного запуска ядра в одном процессе» — для исключения шума от запуска динамического загрузчика; фиксировалось минимальное значение из трёх серий. Машина: AMD Ryzen 7 3700U, 8 ГБ RAM, без GPU.

Таблица 5: Время исполнения `vadd_custom` при различных N (медиана по 50 повторениям)

N	Без проф., мс	Со сбором проф., мс	Накл. расходы, %
256	0.44	1.02	132
1024	1.74	3.89	124
4096	8.34	16.92	103
16384	39.69	74.50	88

Вывод. Накладные расходы профилировщика составляют 88–132% — то есть примерно удваивают время выполнения. Относительные накладные расходы убывают с ростом N : при малом числе инструкций записи в `ProfilingRecord` сопоставимы с суммарным временем исполнения варпов; при больших N исполнение варпов начинает доминировать. Результат согласуется с ожидаемой моделью: накладные расходы профилировщика линейны по числу инструкций, а не по N напрямую.

Двукратное замедление при включённом профилировании приемлемо для задач анализа: детальные метрики уровня инструкций недоступны в обычном режиме.

8.2.2 `gelu_custom` — функция активации GELU

Описание теста. Ядро применяет GELU-аппроксимацию:

$$\text{GELU}(x) = x \cdot \frac{1}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715x^3) \right) \right)$$

к массиву из $N = 4096$ значений. CPU-эталон — `std::tanhf`; допуск — относительная погрешность 10^{-4} . Тест верифицирует трансцендентные инструкции PTX (`ex2`, `lg2`, `fma`) и преобразования типов.

Тестирование. Фиксируются: максимальная относительная ошибка, число инструкций PTX в теле ядра (`Module::Dump()`), наличие дивергенции (ожидается $\eta_{\text{branch}} = 1.0$ — все пути конвергентны).

Вывод. Подтверждает корректность IEEE 754 для трансцендентных функций и демонстрирует линейный рост времени с числом инструкций на поток.

8.2.3 `softmax_custom` — онлайн-нормализация Softmax

Описание теста. Ядро реализует трёхфазный онлайн-алгоритм Softmax: поиск максимума, суммирование экспонент, нормализация. Размер: 1 блок $\times$ 32 потока, длина вектора = 32 элемента. CPU-эталон — двухпроходная реализация; допуск 10^{-4} .

Тест чувствителен к корректности `bar.sync` между фазами и предикатным инструкциям в операциях редукции.

Тестирование. Проверяются: код возврата, сумма выходного вектора ≈ 1.0 (свойство Softmax), максимальная ошибка по элементам. Через профилировщик: число вызовов `bar.sync` (ожидается 2) и $\bar{\eta}_{\text{branch}}$ инструкций редукции.

Вывод. Подтверждает корректность барьерной синхронизации и операций с разделяемой памятью в многофазных редукциях.

8.2.4 `mmul_custom` — тайловое матричное умножение

Описание теста. Умножение матриц $A(M \times K) \times B(K \times N) = C(M \times N)$ с тайловым алгоритмом (тайл 16×16). Размер задачи: $M = N = K = 64$, 16 блоков по 256 потоков. CPU-эталон — наивный тройной цикл; допуск — относительная погрешность 10^{-3} .

Тестирование. Через профилировщик: число уникальных РС в ядре, $\bar{C}_{\text{bank}}$ для `ld.shared` (при правильном заполнении ожидается 0).

Таблица 6: Время исполнения `mmul_custom` (медиана по 5 повторениям, мс)

$M = N = K$	Без проф., мс	Со сбором проф., мс	Накл. расходы, %
32	4.36	28.98	565
64	24.22	191.87	692
96	69.57	606.19	771
128	146.97	1375.96	836

Высокие накладные расходы (565–836%) объясняются плотностью инструкций: каждый поток выполняет K итераций скалярного произведения ($\Theta(K)$ РТХ-инструкций), профилировщик создаёт запись на каждое исполнение, а число записей растёт как $M^3/\text{WARP_SIZE}$.

Вывод. Наиболее ресурсоёмкий тест — верхняя граница оценки производительности эмулятора. Ожидается $O(N^3)$ -рост числа инструкций.

8.2.5 `attention_custom` — Flash Attention-подобное ядро

Описание теста. Ядро реализует:

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

с разбивкой на тайлы по последовательности. Структура управляющего потока: вложенные циклы по тайлам, несколько барьеров, смешанные обращения к разделяемой и глобальной памяти. Верификация против наивной CPU-реализации; допуск 10^{-3} .

Тестирование. Анализируется: глубина стека дивергенции (ожидается $k \leq 2$), число вызовов `bar.sync` на блок, $\bar{\eta}_{\text{branch}}$ внутри тела тайлового цикла.

Вывод. Успешное прохождение подтверждает корректность полного стека: перехват, анализатор, исполнители, PDOM, барьер, разделяемая память. Является главным системным тестом.

8.2.6 `fft_custom` — Radix-2 DIT FFT, $N = 32$

Описание теста. Быстрое преобразование Фурье (алгоритм Cooley-Tukey), $N = 32$, 32 серии. Каждый блок из 32 потоков обрабатывает одну серию: перестановка с обращением битов при загрузке, 5 стадий butterfly, барьеры между стадиями. Верификация против CPU-DFT: $rel_err(k) \leq 10^{-3}$ для всех частот и серий.

Тестирование. Из профилировщика: число инструкций в одном проходе butterfly, $\bar{\eta}_{branch}$ на операции с разделяемой памятью.

Вывод. Подтверждает корректность PDOM при 5 последовательных `__syncthreads`; устранение аварийного завершения анализатора при PTX-файле ≈ 35 КБ.

8.2.7 `conjugate_gradient` — итерационное решение СЛАУ

Описание теста. Метод сопряжённых градиентов для $Ax = b$. Единственный тест с *динамически изменяемым* числом итераций: завершение при $\|r\| < \epsilon$. Верификация: $\|Ax^* - b\|/\|b\| \leq 10^{-4}$.

Тестирование. Из профилировщика: общий `exes_count` суммирующих инструкций (меняется от запуска к запуску), $\bar{\eta}_{branch}$ условия выхода.

Вывод. Единственный тест с нефиксированным числом итераций; подтверждает корректность для итерационных алгоритмов с динамической дивергенцией.

8.2.8 Сводная таблица результатов интеграционных тестов

Таблица 7: Корректность интеграционных тестов

Ядро	Макс. ошибка	PTX инстр.
<code>vadd</code>	$< 10^{-6}$	21
<code>gelu</code>	$< 10^{-4}$	45
<code>softmax</code>	$< 10^{-4}$	54
<code>mmul</code>	$< 10^{-3}$	77
<code>attention</code>	$< 10^{-3}$	129
<code>fft</code>	$< 10^{-3}$	219
<code>conj_grad</code>	$< 10^{-4}$	187

Результаты всех ядер численно совпадают с CPU-эталоном при жёстких порогах погрешности 10^{-3} – 10^{-6} .

Сравнение с V100-PCIe. Для ядер с нетривиальным временем на V100 (`attention` — 132 мкс, `conj_grad` — 476 мкс) замедление сходится к $1\,900$ – $2\,300\times$ —

Таблица 8: Производительность на входных данных ≈ 64 КБ. Сравнение с реальными GPU (V100-PCIE)

Ядро	Без проф. (мс)	Со сбором (мс)	НР (%)	V100 (мкс)	Замедл. ($\times$)
vadd (N=8192)	20.5	34.8	70	7.2	≈ 2800
gelu (N=16384)	46.8	121.8	160	8.0	≈ 5900
softmax (512 $\times$ 32)	60.9	268.4	341	7.8	≈ 7800
mmul (128 $\times$ 128)	111.7	970.5	769	17.8	≈ 6300
attention (S=D=128)	300.5	2074.1	590	131.9	≈ 2300
fft (N=32,B=256)	52.6	218.0	315	8.9	≈ 5900
conj_grad (N=4096)	890.6	1879.7	111	476.3	≈ 1900

НР — накладные расходы профилировщика (%).

это реальная стоимость программной эмуляции PTX без JIT-компиляции и без параллелизма между варпами. Для лёгких ядер (vadd, gelu, softmax, fft) замедление составляет $5\,900\text{--}7\,800\times$: V100 исполняет такие ядра за 7–9 мкс вне зависимости от объёма вычислений, поскольку доминирует задержка запуска ядра (≈ 7 мкс на V100-PCIE), тогда как эмулятор тратит десятки миллисекунд на интерпретацию всех инструкций. Итоговое отношение объясняется малым абсолютным временем GPU, а не показателем превышения ожидаемого замедления.

8.3 Часть II — Тестирование системы профилирования

Методология. Каждый тест запускается с флагом `-collect-profiling`; полученный `profiling.txt` передаётся в `python -m tools.ptx_report`; значения HTML-отчёта сравниваются с аналитически ожидаемыми.

Тесты профилировщика намеренно отделены от функциональных: ядро может давать численно верный результат при ошибочном подсчёте метрик. Каждая метрика проверяется независимо на простом ядре с заранее известным аналитическим значением.

8.3.1 Точность метрик

branch_efficiency: ядро `branch_divergence.cu` (нечётные потоки в одной ветке, чётные — в другой) даёт $\eta = 16/32 = 0.5$; отклонение нулевое, что подтверждает формулу $\eta = \text{popcount}(M)/32$ и запись маски в `ProfilingRecord.write_ops`: ядро `vadd_custom` (3 инструкции с результатом, 1 варп, полная маска) даёт `write_ops[%f] = 32`, что подтверждает детектирование `dst_reg_prefix`. Результаты для `bank_conflicts` и `global_coalescing` сведены в таблицах 9 и 10; строка `f64` ($T_{\text{ideal}} = 2$) подтверждает учёт размера элемента в формуле $T_{\text{ideal}} = \lceil \text{active} \cdot s / 128 \rceil$.

8.3.2 Привязка инструкций к исходному коду

Описание. Ядро `fft_custom.cu` компилируется с `nvcc -lineinfo`. В HTML-отчёте каждая PTX-инструкция аннотирована строкой `fft_custom.cu:L`. Выборочно проверяют-

Таблица 9: Результаты теста `bank_conflicts`

Шаблон	$C_{\text{bank,expected}}$	$C_{\text{bank,emu}}$
Шаг 4 В	0	0
Шаг 128 В	31	31
Шаг 8 В	1	1

Таблица 10: Результаты теста `global_coalescing`

Шаблон	T_{ideal}	κ_{exp}	$T_{\text{actual,emu}}$	κ_{emu}
Последовательный, f32	1	1.000	1	1.000
Шаг 128 Б, f32	1	0.031	32	0.031
Шаг 16 Б, f32	1	0.250	4	0.250
Широковещательный	1	1.000	1	1.000
Последовательный, f64	2	1.000	2	1.000

ся 5 инструкций с известными `.loc`-значениями.

Вывод. Подтверждает корректность разбора `.file/.loc` в `ptx_parser.py`.

8.3.3 Сквозная проверка цепочки профилировщика

Описание. Ядро `vadd_custom` с полностью известной структурой. Аналитически: $\text{exec_count} = 3 \times 32 = 96$, $\bar{\eta}_{\text{branch}} = 1.0$, $C_{\text{bank}} = 0$, $\text{global_mem_transactions} = 2$.

Таблица 11: Сквозная проверка цепочки профилировщика

Метрика	Ожидаемое	HTML-отчёт
<code>exec_count</code>	96	96
$\bar{\eta}_{\text{branch}}$	1.000	1.000
<code>bank_conflicts_total</code>	0	0
<code>global_mem_txns</code>	2	2

Вывод. Нулевые отклонения подтверждают корректность всей цепочки: `profiling.txt` $\rightarrow$ `parser.py` $\rightarrow$ `aggregator.py` $\rightarrow$ `renderer.py`.

9 Заключение

В работе представлен программный эмулятор CUDA PTX, решающий все семь поставленных задач. Ключевые возможности реализации включают: перехват вызовов CUDA Runtime через механизм LD_PRELOAD без модификации бинарного файла приложения; однопроходный инкрементный синтаксический анализатор PTX ISA 7.x [?] с поддержкой 66 мнемоник и автогенерацией регулярных выражений из `instructions.json`; таблицу диспетчеризации с $O(1)$ -поиском и исполнители для всех реализованных инструкций; алгоритмы PDOM_Diverge и PDOM_Merge, обеспечивающие корректную обработку дивергентных потоков внутри варпа; профилировщик с четырьмя метриками и HTML-отчётом с аннотацией строк исходного .cu-файла. Все 7 интеграционных ядер пройдены при максимальном отклонении от CPU-эталоны не более 10^{-3} ; замедление относительно V100-PCIe составляет $1\,900\text{--}2\,300\times$ для нетривиальных нагрузок — неизбежная стоимость программной эмуляции без JIT-компиляции. Для задач непрерывной интеграции и однократного анализа узких мест замедление в $1\,900\text{--}2\,300\times$ приемлемо: прогон эмулятора укладывается в секунды при GPU-времени нетривиальных ядер в сотни микросекунд.

Основное текущее ограничение — неполнота покрытия PTX ISA: ряд инструкций не реализован вовсе (тензорные операции `mma` и `ldmatrix`, часть векторных и специальных функций), а у реализованных инструкций поддержаны не все варианты модификаторов и режимов адресации, предусмотренных спецификацией. Эмулятор прошёл проверку на учебных примерах курса «Программирование на новых архитектурах» и на лабораторных работах студентов, однако остаётся первой версией инструмента. Для практического применения на произвольных промышленных кодах он требует существенного расширения покрытия инструкций, доработки обработки крайних случаев и повышения производительности; первым шагом станет покрытие тензорных инструкций `mma` и `ldmatrix`.

А Таблица поддерживаемых инструкций RTX

Таблица 12: Полная таблица поддерживаемых инструкций RTX

Мнемоника	Поддерживаемые ты	ти-	Режимы округле- ния	Модульный тест
Арифметические				
add	s16/s32/s64, u16/u32/u64, f32/f64	—	—	test_executors
sub	s16/s32/s64, u16/u32/u64, f32/f64	—	—	test_executors
mul	s16/s32, u16/u32, f32/f64	—	rn/rz/ru/rd (f)	test_executors
div	s32/u32/s64/u64, f32/f64	—	rn/rz/ru/rd (f)	test_executors
rem	s32/u32/s64/u64	—	—	test_executors
abs	s16/s32/s64, f32/f64	—	—	test_executors
neg	s16/s32/s64, f32/f64	—	—	test_executors
min	s32/u32/s64/u64, f32/f64	—	—	test_executors
max	s32/u32/s64/u64, f32/f64	—	—	test_executors
fma	f32/f64	—	rn/rz/ru/rd	test_executors
mad	s32/u32	—	—	test_executors
Логические и битовые				
and	b32/b64, pred	—	—	test_executors
or	b32/b64, pred	—	—	test_executors
xor	b32/b64, pred	—	—	test_executors
not	b32/b64, pred	—	—	test_executors
shl	b32/b64	—	—	test_executors
shr	b32/b64, s32/u32/s64/u64	—	—	test_executors
popc	b32/b64	—	—	test_executors
bfe	u32/s32/u64/s64	—	—	test_executors
bfi	b32/b64	—	—	test_executors
Сравнения и предикаты				
setp	s32/u32/s64/u64, f32/f64	—	—	test_executors
selp	s32/f32/s64/f64	—	—	test_executors
slct	s32/f32	—	—	test_executors
Пересылки и память				
mov	b32/b64, u32/u64, f32/f64, pred	—	—	test_executors
ld	b8-b128, f32/f64 / all spaces	—	—	test_executors
st	b8-b128, f32/f64 / all spaces	—	—	test_executors
cvta	u64 (to/from generic)	—	—	test_executors
Преобразования типов				
cvt	все пары {s8..f64}	—	rn/rz/ru/rd/ru	test_executors

Таблица 12 — продолжение

Мнемоника	Поддерживаемые пы	ти-	Режимы округле- ния	Модульный тест
Управление потоком				
bra	—		—	test_barrier
call	—		—	test_barrier
ret	—		—	test_barrier
exit	—		—	test_barrier
Синхронизация и специальные				
bar.sync	—		—	test_barrier
membar	cta/gl/sys		—	test_barrier
ex2	f32		—	test_executors
lg2	f32		—	test_executors
rcp	f32/f64		rn/rz/ru/rd	test_executors
sqrt	f32/f64		rn/rz	test_executors
sin	f32		—	test_executors
cos	f32		—	test_executors

В Поведение инструкций РТХ: полный список правил

Приложение содержит формальные правила поведения для всех реализованных инструкций. Используются обозначения: M — маска исполнения, $R[t][\tau][k]$ — значение регистра k типа τ потока t , $\text{active}(M) = \{i : M[i] = 1\}$.

Арифметические инструкции

`add.s32 %d, %a, %b:`

$$R'[t][s32][d] = (R[t][s32][a] + R[t][s32][b]) \bmod 2^{32}, \quad \forall t \in \text{active}(M).$$

`sub.u64 %d, %a, %b:`

$$R'[t][u64][d] = (R[t][u64][a] - R[t][u64][b]) \bmod 2^{64}.$$

`mul.lo.s32 %d, %a, %b:`

$$R'[t][s32][d] = (R[t][s32][a] \cdot R[t][s32][b]) \bmod 2^{32}.$$

`mul.hi.u32 %d, %a, %b:`

$$R'[t][u32][d] = \left\lfloor \frac{R[t][u32][a] \cdot R[t][u32][b]}{2^{32}} \right\rfloor.$$

`fma.rn.f32 %d, %a, %b, %c:`

$$R'[t][f32][d] = \text{round}_n(R[t][f32][a] \cdot R[t][f32][b] + R[t][f32][c]).$$

`rcp.rn.f32 %d, %a:`

$$R'[t][f32][d] = \text{round}_n\left(\frac{1}{R[t][f32][a]}\right).$$

Инструкции сравнения

`setp.lt.s32 %p, %a, %b:`

$$P'[t] = (R[t][s32][a] < R[t][s32][b]), \quad \forall t \in \text{active}(M).$$

`setp.gt.f32 %p|%q, %a, %b:`

$$P'[t] = (R[t][f32][a] > R[t][f32][b]), \quad Q'[t] = \neg P'[t].$$

`selp.f32 %d, %a, %b, %p:`

$$R'[t][f32][d] = \begin{cases} R[t][f32][a] & \text{если } P[t] = 1, \\ R[t][f32][b] & \text{если } P[t] = 0. \end{cases}$$

Инструкции памяти

`ld.global.f32 %d, [%a]:`

$$R'[t][f32][d] = Mem_{\text{global}}[R[t][u64][a]].$$

`st.shared.f32 [%a], %b:`

$$Mem'_{\text{shared}}[R[t][u64][a]] = R[t][f32][b], \quad \forall t \in \text{active}(M).$$

Инструкции управления потоком

`bra.uni $L:`

$$\langle \text{bra.uni } \$L, \sigma \rangle \rightarrow \sigma[\text{PC} \leftarrow \text{offset}(\$L)].$$

`ret:`

$$\langle \text{ret}, \sigma \rangle \rightarrow \sigma[\text{PC} \leftarrow pc_return, cur_func \leftarrow caller_func].$$

С Трассировка PDOM_Diverge на примере вложенной дивергенции

Рассмотрим фрагмент CUDA-кода с двумя вложенными условными ветвлениями:

Листинг 18: Пример вложенной дивергенции

```
// tid = threadIdx.x
if (tid < 16) {           // outer if: threads 0-15 / 16-31
    if (tid < 8) {        // inner if: threads 0-7 / 8-15
        A();
    } else {
        B();
    }
    C();
} else {
    D();
}
E(); // reconvergence point
```

Начальная маска: $M_0 = 0xFFFFFFFF$. Базовые блоки: `bb_outer_if`, `bb_inner_if`, `bb_A`, `bb_B`, `bb_C`, `bb_D`, `bb_E` (реконвергенция).

Пусть $PC_E = 100$ — РС метки `E`, $PC_C = 50$ — РС метки `C`.

Трассировка подтверждает, что LIFO-порядок снятия фреймов восстанавливает маски в порядке «внутреннее-первым», затем «внешнее-первым», что обеспечивает корректное исполнение всех ветвей при произвольной глубине вложенности.

Таблица 13: Трассировка стека дивергенции

Шаг	Текущий РС / инструкция	Стек S (вершина справа)
1	setp.lt + @%p bra \$outer	Внешняя дивергенция: $M_{\text{taken}} = 0x0000FFFF$, $M_{\text{not}} = 0xFFFF0000$ push конвергенции $\{100, 0xFFFFFFFF\}$, push «не взята» $\{PC_D, 0xFFFF0000\}$ $S = [\{100, M_0, T\}, \{PC_D, M_{\text{not}}, F\}]$
2	Исполнение bb_inner_if: @%p bra \$inner Внутренняя дивергенция:	$M = 0x0000FFFF$; $M_{\text{tk}} = 0x000000FF$, $M_{\text{nt}} = 0x0000FF00$ push $\{PC_C, 0x0000FFFF, T\}$, push $\{PC_B, 0x0000FF00, F\}$ $S = [\{100, M_0, T\}, \{PC_D, M_{\text{not_outer}}, F\},$ $\{PC_C, 0x0000FFFF, T\}, \{PC_B, 0x0000FF00, F\}]$
3	Исполнение bb_A ($M = 0x000000FF$)	S без изменений
4	РС = PC_B (цикл реконвергенции)	Снимаем $\{PC_B, 0x0000FF00, F\}$: $M \leftarrow 0x0000FF00$, $S =$ $[\{100, M_0, T\}, \{PC_D, M_{\text{not}}, F\}, \{PC_C, 0x0000FFFF, T\}]$
5	Исполнение bb_B ($M = 0x0000FF00$)	S без изменений
6	РС = PC_C (цикл реконвергенции)	Снимаем $\{PC_C, 0x0000FFFF, T\}$: $M \leftarrow 0x0000FFFF$, $S =$ $[\{100, M_0, T\}, \{PC_D, M_{\text{not}}, F\}]$
7	Исполнение bb_C ($M = 0x0000FFFF$)	S без изменений
8	РС = PC_D (цикл реконвергенции)	Снимаем $\{PC_D, M_{\text{not}}, F\}$: $M \leftarrow 0xFFFF0000$, $S = [\{100, M_0, T\}]$
9	Исполнение bb_D ($M = 0xFFFF0000$)	S без изменений
10	РС = 100 (цикл реконвергенции)	Снимаем $\{100, M_0, T\}$: $M \leftarrow 0xFFFFFFFF$, $S = []$
11	Исполнение bb_E ($M = 0xFFFFFFFF$)	Полная реконвергенция

D Формат файла profiling.txt: аннотированный пример

Файл `profiling.txt` имеет иерархическую структуру: «запуск ядра → функция → базовый блок → записи инструкций». Ниже приведён аннотированный фрагмент для ядра `fft_custom`.

Листинг 19: Пример `profiling.txt` для `fft_custom`

```
Launch 0 _Z10fft_kernelPcfi
Function _Z10fft_kernelPcfi
Basic Block $BBO_1
[00:01:23.456] [PC: 0x000000000000002a] Block: [0, 0, 0] WarpId: 0 Execution Mask
0xffffffff ld.global.f32 branch_efficiency: 1.000000, global_mem_transactions:
1, global_coalescing: 1.000000
[00:01:23.456] [PC: 0x000000000000002b] Block: [0, 0, 0] WarpId: 0 Execution Mask
0xffffffff fma.rn.f32 branch_efficiency: 1.000000
Basic Block $BBO_3
[00:01:23.457] [PC: 0x000000000000003d] Block: [0, 0, 0] WarpId: 0 Execution Mask
0x0000ffff bra branch_efficiency: 0.500000
Basic Block $BBO_5
[00:01:23.457] [PC: 0x000000000000003e] Block: [0, 0, 0] WarpId: 0 Execution Mask
0x0000ffff st.shared.f32 branch_efficiency: 0.500000, bank_conflicts: 0
Basic Block $BBO_7
[00:01:23.458] [PC: 0x000000000000004b] Block: [0, 0, 0] WarpId: 0 Execution Mask
0xffffffff bar branch_efficiency: 1.000000
```

Описание полей записи:

- [HH:MM:SS.mmm] — монотонная временная метка (формируется `Profiler::FormatTimestamp`).
- [PC: 0хNNNN...] — шестнадцатеричный монотонный РС инструкции (16 символов, дополненных нулями).
- Block: [x, y, z] — трёхмерный идентификатор блока.
- WarpId: N — идентификатор варпа внутри блока.
- Execution Mask 0хNNNNNNNN — маска активных потоков (32 бита, 8 шестнадцатеричных символов).
- instr_name — мнемоника инструкции.
- metric: value, ... — список метрик в формате **имя: значение**, разделённых запятой.

Описание заголовочных строк:

- Launch N func_name — начало нового запуска ядра (N — монотонный счётчик).
- Function func_name — начало записей для новой RTX-функции.
- Basic Block \$BBx_y — начало записей для нового базового блока.

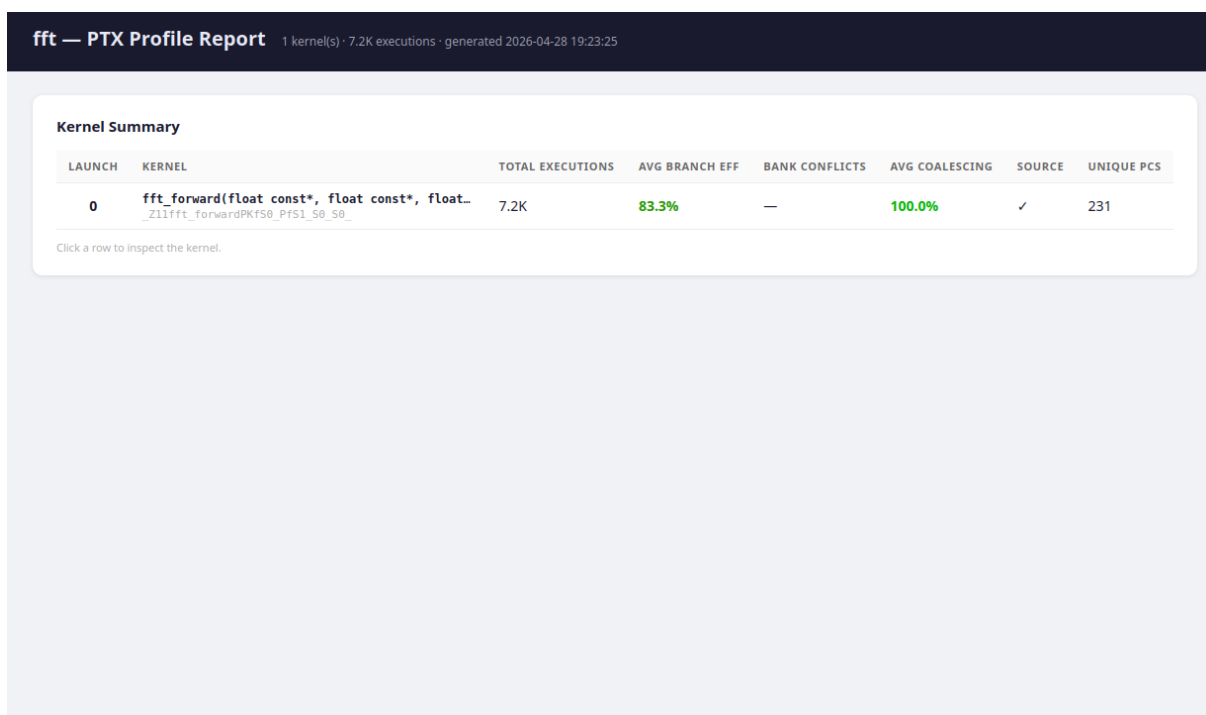
Е Структура HTML-отчёта: описание секций

HTML-отчёт генерируется командой:

```
python -m tools.ptx_report \
  --profiling profiling.txt \
  --ptx fft_kernel.ptx \
  --output report_fft.html
```

Отчёт содержит пять секций:

Секция 1 — Панель «пилюль» (рисунок 4): сводные метрики ядра — общее число инструкций, число исполненных потоко-операций, среднее $\bar{\eta}_{\text{branch}}$, суммарные C_{bank} , средний $\bar{\kappa}$.



LAUNCH	KERNEL	TOTAL EXECUTIONS	AVG BRANCH EFF	BANK CONFLICTS	AVG COALESCING	SOURCE	UNIQUE PCS
0	fft_forward(float const*, float const*, float..., _z11fft_forwardPKfs0_Pfs1_50_50_)	7.2K	83.3%	—	100.0%	✓	231

Click a row to inspect the kernel.

Рисунок 4: Панель «пилюль» HTML-отчёта.

Секция 2 — Таблица инструкций (рисунок 5): все уникальные РС с тепловой подсветкой по `exes_count`; столбцы: РС, мнемоника, блок, метрики.

Секция 3 — Горячие точки: топ-10 инструкций по дивергенции (минимальный η_{branch}), по конфликтам банков (максимальный C_{bank}), по слиянию (минимальный κ).

Секция 4 — Таблица регистров: для каждого префикса регистров — тип, объявлено слотов, накопленный `write_ops`.

Секция 5 — Аннотированный исходный код: строки исходного `.cu`-файла с вложенными метриками инструкций, привязанных через `.loc`-директивы.

PTX ASSEMBLY					
PC	PTX Instruction	Exec Count	Branch Eff	Mem Trans	Coalescing
0x001e	or.b32 %r20, %r19, %r16;	32	100.0%	—	—
fft_c_					
0x001f	or.b32 %r21, %r20, %r1;	32	100.0%	—	—
fft_c_					
0x0020	cvta.to.global.u64 %rd9, %rd5;	32	100.0%	—	—
fft_c_					
0x0021	mul.wide.s32 %rd10, %r21, 4;	32	100.0%	—	—
0x0022	add.s64 %rd11, %rd9, %rd10;	32	100.0%	—	—
0x0023	ld.global.f32 %f56, [%rd11];	32	100.0%	32	100.0%
0x0024	mov.u32 %r22, _ZZ11fft_forwardPKfs0_Pfs1_...	32	100.0%	—	—
0x0025	add.s32 %r8, %r22, %r13;	32	100.0%	—	—
0x0026	st.shared.f32 [%r8], %f56;	32	100.0%	—	—
fft_c_					
0x0027	cvta.to.global.u64 %rd12, %rd6;	32	100.0%	—	—
fft_c_					
0x0028	add.s64 %rd13, %rd12, %rd10;	32	100.0%	—	—
0x0029	ld.global.f32 %f57, [%rd13];	32	100.0%	32	100.0%
0x002a	mov.u32 %r23, _ZZ11fft_forwardPKfs0_Pfs1_...	32	100.0%	—	—
0x002b	add.s32 %r9, %r23, %r13;	32	100.0%	—	—
0x002c	st.shared.f32 [%r9], %f57;	32	100.0%	—	—
fft_c_					
0x002d	bar.sync 0;	32	100.0%	—	—
fft_c_					
0x002e	xor.b32 %r24, %r13, 4;	32	100.0%	—	—
0x002f	add.s32 %r25, %r22, %r24;	32	100.0%	—	—
fft_c_					
0x0030	add.s32 %r26, %r23, %r24;	32	100.0%	—	—
fft_c_					
0x0031	ld.shared.f32 %f1, [%r25];	32	100.0%	—	—

Рисунок 5: Таблица инструкций с тепловой подсветкой.

SOURCE				PTX ASSEMBLY					
Line	Source	PTX	Exec	PC	PTX Instruction	Exec Count	Branch Eff	Mem Tra	Coalescing
36	const int half = 1 << s;			fft_c_					
37	const int partner = tid ^ half;			0x0038	ld.shared.f32 %f6, [...]	32	100.0%	—	—
38	const int j = tid & (half - 1);			fft_c_					
39	const int is_lower = (tid >> s) & 1;			0x0039	ld.shared.f32 %f7, [...]	32	100.0%	—	—
40				fft_c_					
41	const float wr = tw_r[s * (FFT_N / 2) + j];	14	448	0x003a	mul.f32 %f59, %f2, %...	32	100.0%	—	—
42	const float wi = tw_i[s * (FFT_N / 2) + j];	9	288	0x003b	fma.rm.f32 %f8, %f4, ...	32	100.0%	—	—
43				fft_c_					
44	const float a_r = sr[tid];	5	160	0x003c	bar.sync 0;	32	100.0%	—	—
45	const float a_i = si[tid];	5	160	fft_c_					
46	const float b_r = sr[partner];	15	480	0x003d	setp.eq.s32 %p1, %r3, ...	32	100.0%	—	—
47	const float b_i = si[partner];	10	320	0x003e	@p1 bra \$L_BB0_2;	32	50.0%	—	—
48				0x003f	bra.uni \$L_BB0_1;	32	50.0%	—	—
49	const float tb_r = wr * b_r - wi * b_i;	15	480	fft_c_					
50	const float tb_i = wr * b_i + wi * b_r;	10	320	0x0040	mul.f32 %f64, %f4, %...	32	50.0%	—	—
51	const float ta_r = wr * a_r - wi * a_i;	15	480	0x0041	mul.f32 %f65, %f2, %...	32	50.0%	—	—
52	const float ta_i = wr * a_i + wi * a_r;	10	320	0x0042	sub.f32 %f66, %f65, ...	32	50.0%	—	—
53				fft_c_					
54	__syncthreads();	5	160	0x0043	add.f32 %f67, %f6, %...	32	50.0%	—	—
55				0x0044	st.shared.f32 [%r8], ...	32	50.0%	—	—
56	if (!is_lower)	15	480	fft_c_					
57	{			0x0045	add.f32 %f110, %f7, ...	32	50.0%	—	—
58	sr[tid] = a_r + tb_r;	10	320	0x0046	bra.uni \$L_BB0_3;	32	50.0%	—	—
59	si[tid] = a_i + tb_i;	10	320	fft_c_					
60	}			0x0047	mul.f32 %f60, %f4, %...	32	50.0%	—	—
61	else			0x0048	mul.f32 %f61, %f2, %...	32	50.0%	—	—
62	{			0x0049	sub.f32 %f62, %f61, ...	32	50.0%	—	—
63	sr[tid] = b_r - ta_r;	10	320	fft_c_					
64	si[tid] = b_i - ta_i;	5	160	0x004a	sub.f32 %f63, %f1, %...	32	50.0%	—	—

Рисунок 6: Аннотированный исходный код в HTML-отчёте.

Г Репозиторий с исходным кодом

Исходный код эмулятора CUDA PTX, тесты и вспомогательные скрипты опубликованы в открытом репозитории на платформе GitHub:

`https://github.com/Poolce/ptx-emulator`